
COMPLETE STUDY GUIDE

Data Structures & Algorithms

18 Lectures - Complete Course Coverage
With Code, Dry Runs, Flowcharts & MCQs

Tehreem Rana

STUDENT

Covering: Arrays, Linked Lists, Stacks, Queues, Trees, Graphs, BST, AVL Trees, Bubble Sort, Selection Sort, Insertion Sort, Radix Sort, Merge Sort, Quick Sort, Heap Sort, Linear Search, Binary Search, Dijkstra's & Bellman-Ford Algorithms, Tree Traversals, AVL Rotations (LL, RR, LR, RL), Prefix/Postfix Notation



Table of Contents

Chapter 1: Introduction to Data Structures & Algorithms	9
What is Data?	9
Data Types	9
What is a Data Structure?	9
Classification of Data Structures	10
Primitive Data Structures	11
Non-Primitive (Abstract) Data Structures	11
1. Linear Data Structures	11
2. Non-Linear Data Structures	12
What is an Algorithm?	12
Why are DSA Needed?	12
Key Terminology	12
Data Structure Operations	13
Time Complexity Comparison	13
Real-Life Examples of Data Structures	14
Important Points - Chapter 1	14
Practice Questions - Chapter 1	14
Chapter 2: Arrays & Bubble Sort	15
Array Overview	15
Types of Arrays	16
Array Operations & Complexity	16
Real-Life Example of Array	16
C++ Code: Array Operations	17
Bubble Sort	17
How Bubble Sort Works	17

Algorithm Steps	18
Flowchart - Bubble Sort	18
C++ Code: Bubble Sort	18
Dry Run: Bubble Sort	19
Passes and Steps Formula	20
Time Complexity - Bubble Sort	20
Advantages & Disadvantages	21
Applications	21
Important Points - Chapter 2	21
Practice Questions - Chapter 2	22
Chapter 3: Selection Sort	23
Algorithm Steps	23
Flowchart - Selection Sort	23
C++ Code: Selection Sort	24
Dry Run: Selection Sort	24
Passes and Steps Formula	25
Time Complexity - Selection Sort	25
Advantages & Disadvantages	26
Important Points - Chapter 3	26
Practice Questions - Chapter 3	26
Chapter 4: Insertion Sort	27
Algorithm Steps	28
C++ Code: Insertion Sort	28
Dry Run: Insertion Sort	28
Time Complexity - Insertion Sort	29
Advantages & Disadvantages	29
Important Points - Chapter 4	30

Practice Questions - Chapter 4	30
Chapter 5: Radix Sort	31
Algorithm Steps	31
C++ Code: Radix Sort	31
Dry Run: Radix Sort	32
Time Complexity	33
Advantages & Disadvantages	34
Important Points - Chapter 5	34
Practice Questions - Chapter 5	34
Chapter 6: Linear Search	35
Algorithm Steps	35
C++ Code: Linear Search	36
Dry Run: Linear Search	36
Time Complexity - Linear Search	36
Advantages & Disadvantages	37
Important Points - Chapter 6	37
Practice Questions - Chapter 6	37
Chapter 7: Binary Search	38
Binary Search Conditions	39
Algorithm Steps	39
C++ Code: Binary Search	39
Dry Run: Binary Search	40
Time Complexity - Binary Search	40
Advantages & Disadvantages	41
Important Points - Chapter 7	41
Practice Questions - Chapter 7	41

Chapter 8: Merge Sort	42
How Merge Sort Works	43
C++ Code: Merge Sort	43
Dry Run: Merge Sort	44
Time Complexity - Merge Sort	44
Advantages & Disadvantages	45
Important Points - Chapter 8	45
Practice Questions - Chapter 8	46
Chapter 9: Quick Sort	47
Choice of Pivot Element	47
C++ Code: Quick Sort	47
Dry Run: Quick Sort	48
Time Complexity - Quick Sort	48
Advantages & Disadvantages	49
Important Points - Chapter 9	49
Practice Questions - Chapter 9	49
Chapter 10: Heap Sort	50
What is a Binary Heap?	51
Working Process	51
C++ Code: Heap Sort	51
Dry Run: Heap Sort	52
Time Complexity - Heap Sort	53
Advantages & Disadvantages	53
Important Points - Chapter 10	54
Practice Questions - Chapter 10	54
Chapter 11: Linear Data Structures	55
1. Array	55

2. Linked List	55
Types of Linked Lists	56
C++ Code: Singly Linked List	56
3. Stack	57
C++ Code: Stack using Array	58
4. Queue	59
Types of Queues	59
C++ Code: Queue using Array	59
Operations Comparison	60
Important Points - Chapter 11	60
Practice Questions - Chapter 11	61
Chapter 12: Trees & Graphs	61
Tree Data Structure	62
Tree Terminology	62
Types of Trees	62
Key Tree Operations	63
Applications of Trees	64
Graph Data Structure	64
Types of Graphs	64
Graph Properties	65
Graph Representation	65
Applications of Graphs	65
Important Points - Chapter 12	65
Practice Questions - Chapter 12	66
Chapter 13: Binary Tree & Binary Search Tree	67
Binary Tree	67
Binary Tree Terminology	67

C++ Code: Binary Tree Node	68
Binary Search Tree (BST)	68
BST Operations	68
Search in BST	68
Insert in BST	68
C++ Code: BST Operations	69
Time Complexity - BST Operations	69
Advantages & Disadvantages of BST	70
Important Points - Chapter 13	70
Practice Questions - Chapter 13	70
Chapter 14: Tree Traversal Techniques	71
Types of Traversals	71
Example Tree	72
1. Inorder Traversal (Left -> Root -> Right)	72
2. Preorder Traversal (Root -> Left -> Right)	73
3. Postorder Traversal (Left -> Right -> Root)	73
4. Level Order Traversal (BFS)	73
C++ Code: All Traversals	74
Traversal Summary	75
Important Points - Chapter 14	75
Practice Questions - Chapter 14	75
Chapter 15: Balanced Binary Tree	76
Balance Factor	77
How to Check if a Tree is Balanced	77
C++ Code: Check if Tree is Balanced	78
Advantages of Balanced Trees	78
Important Points - Chapter 15	78

Practice Questions - Chapter 15	79
Chapter 16: AVL Tree	80
AVL Tree Properties	80
AVL Rotations (CRITICAL - Exam Topic!)	80
1. LL Rotation (Left-Left Case)	81
2. RR Rotation (Right-Right Case)	81
3. LR Rotation (Left-Right Case)	82
4. RL Rotation (Right-Left Case)	82
Rotation Summary Table	83
C++ Code: AVL Tree	83
Complete AVL Insertion Dry Run	85
Time Complexity - AVL Tree	85
Advantages & Disadvantages	86
Important Points - Chapter 16	86
Practice Questions - Chapter 16	86
Chapter 17: Shortest Path Algorithms	88
Dijkstra's Algorithm	88
Algorithm Steps	88
Dry Run: Dijkstra's Algorithm	89
Complexity - Dijkstra	89
Bellman-Ford Algorithm	89
Algorithm Steps	90
Complexity - Bellman-Ford	90
Dijkstra vs Bellman-Ford	90
Important Points - Chapter 17	90
Practice Questions - Chapter 17	91
Chapter 18: Prefix / Postfix String Manipulation	91

Types of Notation	92
Algorithm: String Grouping	92
C++ Code: Prefix / Postfix Grouping	92
Dry Run: String Grouping	93
Complexity Analysis	93
Important Points - Chapter 18	93
Practice Questions - Chapter 18	94
 Complete Complexity Summary Table	 95
Sorting Algorithms	95
Searching Algorithms	95
Data Structures & Graph Algorithms	95
 Comprehensive MCQs - All Chapters	 96
 Comprehensive Final Test	 103
Section A: Multiple Choice (2 marks each)	103
Section B: Short Answer Questions (5 marks each)	104
Section C: Long Answer Questions (10 marks each)	105

Chapter 1: Introduction to Data Structures & Algorithms

What is Data?

Data is any information that can be processed by a computer. It includes numbers, text, images, audio, and video. Everything a computer works with is ultimately data. In programming, data is the raw material that we store, manipulate, and transform to produce meaningful results. Without data, there would be nothing for programs to process or for algorithms to work on. Understanding data at its most fundamental level is essential for any computer science student because all higher-level concepts in computing are built upon the foundation of data and how it is organized.

Data Types

A data type defines the kind of value a variable can hold and the operations that can be performed on it. Every programming language provides built-in (primitive) data types. Understanding data types is crucial because it determines how much memory is allocated, what operations are valid, and how the data is interpreted by the compiler or interpreter. Choosing the correct data type prevents errors, saves memory, and improves performance.

Integer: A whole number (positive or negative) without a fractional part. Example: -5, 0, 42.
Real (Float/Double): A number that contains a decimal or fractional part. Example: 3.14, -0.001, 99.99.
Boolean: Stores one of two values: TRUE or FALSE. Used for logical operations and decision-making.
Character: A single letter, digit, punctuation mark, or symbol. Typically occupies one byte of storage using ASCII encoding.
String: A sequence of characters, often called "text." Example: "Hello World". Each character is stored as one byte using its ASCII code, and the length is limited only by available memory.

What is a Data Structure?

Data Structure: A data structure is a specialized format for organizing, processing, retrieving, and storing data efficiently. It defines the relationship between the data elements and the operations that can be performed on them.
--

performed on them.

Data structures are fundamental building blocks in computer science. They allow us to manage large amounts of data efficiently, support fast and powerful algorithms, and organize data for nearly every software system. The choice of data structure directly impacts the performance of a program. A well-chosen data structure can reduce execution time significantly, while a poor choice can make even a simple task painfully slow.

Key Roles of Data Structures:

- **Efficiency:** They allow management of large amounts of data for services like databases and internet indexing services.
- **Algorithm Support:** They are essential for creating fast and powerful algorithms by reducing complexity.
- **Core Functions:** They organize, process, retrieve, and store data for nearly every software system.
- **Optimization:** An efficient structure uses minimum memory and execution time.

Data Structures help to:

- Store large amounts of data efficiently.
- Perform operations like searching, sorting, inserting, and deleting faster.
- Model real-world problems such as networks and hierarchies.
- Optimize the time and space complexity of programs.

Classification of Data Structures

Data structures are broadly categorized into two main types: Primitive Data Structures and Non-Primitive (Abstract) Data Structures. Primitive data structures are the basic building blocks provided by programming languages, such as integers, floats, characters, and booleans. Non-primitive data structures are more complex and are built using primitive types to provide specialized operations. They are further divided into Linear and Non-Linear data structures.

Classification of Data Structures

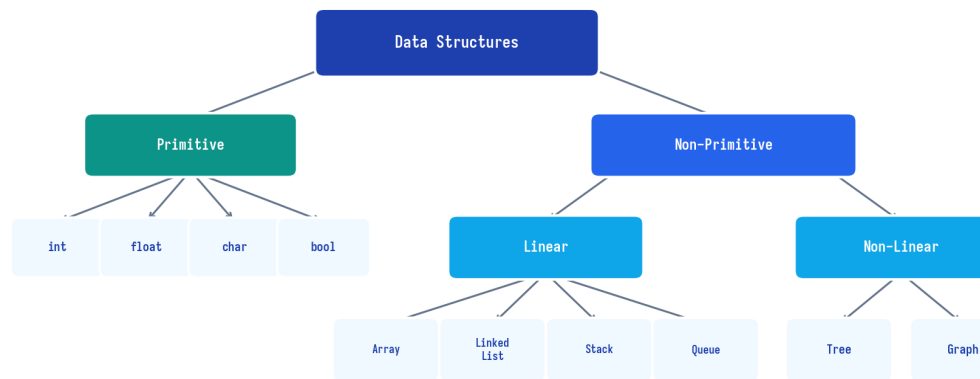


Figure 1.1: Classification of Data Structures

Primitive Data Structures

These are the basic data types provided by programming languages. They represent single values and are directly operated upon by the machine. Examples include Integer, Float, Character, and Boolean. They are simple, fast to access, and require fixed amounts of memory.

Non-Primitive (Abstract) Data Structures

These are more complex structures built from primitive data types. They are designed to organize data in ways that support specific operations efficiently. Non-primitive data structures are divided into two categories:

1. Linear Data Structures

Elements are arranged sequentially in one dimension. Each element has a previous and next adjacent neighbor (except for the first and last elements). Examples include:

- **Array:** Fixed-size collection of same-type elements stored at contiguous memory locations. Applications: Image processing, matrices.
- **Linked List:** Nodes connected using pointers. Applications: Dynamic memory management.
- **Stack (LIFO):** Last In, First Out structure. Applications: Function calls, undo/redo operations.
- **Queue (FIFO):** First In, First Out structure. Applications: CPU scheduling, printer queues.

2. Non-Linear Data Structures

Elements are arranged in hierarchical or graph-like structures where each element can connect to multiple other elements. Examples include:

- Tree: Hierarchical structure with parent-child relationships. Applications: File systems, databases.
- Graph: Collection of nodes (vertices) and edges connecting them. Applications: Social networks, routing.
- Heap: Specialized tree for priority handling. Applications: Priority queues.
- Hash Table: Key-value pair storage with fast lookup. Applications: Caches, symbol tables.

What is an Algorithm?

Algorithm: An algorithm is a step-by-step set of instructions designed to solve a specific problem or achieve a particular goal. It takes some input, processes it through a finite sequence of well-defined steps, and produces an output.

Algorithms are fundamental to programming. An efficient algorithm can transform a slow, unusable program into a fast, responsive one. The study of algorithms involves understanding their correctness, efficiency (time and space complexity), and optimality. A good algorithm should be unambiguous, have well-defined inputs and outputs, be finite (terminate after a finite number of steps), and be feasible (can be executed with available resources).

Why are DSA Needed?

- Managing massive data: Social networks with billions of users, search engines indexing the entire web.
- Scheduling tasks: CPU scheduling, job scheduling in operating systems.
- Planning routes: GPS navigation finding the shortest path.
- Optimizing processes: Packing problems, task completion optimization.
- Solving complex problems: Machine learning, artificial intelligence, scientific computing.

Key Terminology

Term	Description
Algorithm	Step-by-step instructions to solve a specific problem.

Data Structure	A way of organizing data for efficient use.
Time Complexity	Measure of time an algorithm takes based on input size.
Space Complexity	Measure of memory an algorithm uses based on input size.
Big O Notation	Mathematical notation for the upper bound of algorithm complexity.
Recursion	A technique where a function calls itself.
Divide and Conquer	Breaking a problem into smaller sub-problems.
Brute Force	Trying all possible solutions to find the best one.

Table 1.1: Key DSA Terminology

Data Structure Operations

- Insertion: Adding new data or a new node to the structure.
- Deletion: Removing a node from the structure.
- Traversing: Accessing each node exactly once for processing.
- Searching: Finding the location of a node based on a key value.
- Sorting: Arranging data in a particular order (ascending or descending).
- Merging: Joining two data structures together.

Time Complexity Comparison

Time complexity describes how the running time of an algorithm grows as the input size increases. Big O notation is used to express this growth rate. Common time complexities from fastest to slowest are: $O(1)$ (constant), $O(\log n)$ (logarithmic), $O(n)$ (linear), $O(n \log n)$ (linearithmic), $O(n^2)$ (quadratic), and $O(2^n)$ (exponential). Understanding these is crucial for choosing the right algorithm for your problem.

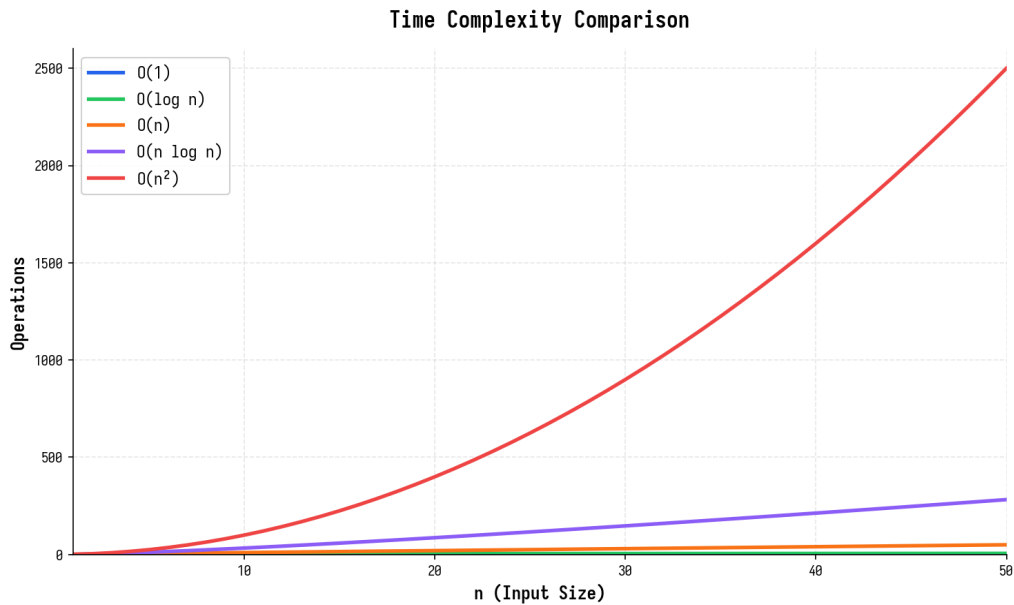


Figure 1.2: Time Complexity Comparison

Real-Life Examples of Data Structures

- Array: A row of lockers, each with a number, storing one item each.
- Linked List: A treasure hunt where each clue points to the next location.
- Stack: A stack of plates in a cafeteria - you add and remove from the top only.
- Queue: People standing in line at a ticket counter - first come, first served.
- Tree: The organizational chart of a company with CEO at the root.
- Graph: A map of cities connected by roads, or a social network of friends.

Important Points - Chapter 1

Key Takeaways:

1. Data is raw information; data structures organize it for efficient use.
2. Primitive DS: int, float, char, bool. Non-Primitive: Array, Linked List, Stack, Queue, Tree, Graph.
3. Linear DS: Elements in sequence. Non-Linear DS: Elements in hierarchy or network.
4. Algorithm = Step-by-step problem-solving instructions.
5. Big O notation measures worst-case complexity.
6. Common operations: Insert, Delete, Traverse, Search, Sort, Merge.
7. Choose data structure based on: type of data, operations needed, and frequency of operations.

Practice Questions - Chapter 1

Q1. Which of the following is a non-linear data structure?

- (A) Array
- (B) Stack
- (C) Tree
- (D) Queue

Answer: C

Q2. What does Big O notation describe?

- (A) Best-case time
- (B) Worst-case time
- (C) Average time
- (D) Exact time

Answer: B

Q3. Which data structure follows the LIFO principle?

- (A) Queue
- (B) Stack
- (C) Array
- (D) Linked List

Answer: B

Q4. A data structure that stores key-value pairs is called:

- (A) Array
- (B) Stack
- (C) Hash Table
- (D) Queue

Answer: C

Q5. Which is NOT a primitive data type?

- (A) Integer
- (B) Boolean
- (C) Array
- (D) Character

Answer: C

Chapter 2: Arrays & Bubble Sort

Array Overview

Array: An array is a linear data structure that stores a fixed-size collection of elements of the same data type at contiguous memory locations. Elements are accessed using an index (position number), starting from index 0.

Arrays are one of the most fundamental and widely used data structures in computer science. They provide constant-time access to any element when the index is known ($O(1)$), making them extremely efficient for read-heavy operations. However, inserting or deleting elements in the middle of an array requires shifting all subsequent elements, which takes $O(n)$ time. Arrays are used as the underlying structure for many other data structures and algorithms.

Types of Arrays

- 1D Array (One-Dimensional): A single row of elements. Example: [10, 20, 30, 40, 50]
- 2D Array (Matrix): A grid of rows and columns. Example: A 3x3 matrix for image pixels.
- Multi-Dimensional Array: Arrays with more than two dimensions. Example: 3D array for RGB image data.

Array Operations & Complexity

Operation	Time Complexity	Description
Access by Index	$O(1)$	Direct access using index position
Search (Unsorted)	$O(n)$	Linear scan to find element
Search (Sorted)	$O(\log n)$	Binary search on sorted array
Insert at End	$O(1)$	Append to last position
Insert at Beginning/Middle	$O(n)$	Shift all elements after insertion point
Delete from End	$O(1)$	Remove last element
Delete from Beginning/Middle	$O(n)$	Shift all elements after deletion point

Table 2.1: Array Operations Complexity

Real-Life Example of Array

Think of a row of mailboxes in an apartment building. Each mailbox has a number (index) and contains letters (data). You can directly go to mailbox #5 without checking mailboxes #1 through #4 (random access). But if you want to add a new mailbox in the middle, you have to shift all the mailboxes after it.

C++ Code: Array Operations

```
#include <iostream>
using namespace std;

int main() {
    // Declare and initialize an array
    int arr[5] = {7, 12, 9, 11, 3};
    int n = 5;

    // Access element by index - O(1)
    cout << "Element at index 2: " << arr[2] << endl;

    // Linear search - O(n)
    int key = 9;
    for (int i = 0; i < n; i++) {
        if (arr[i] == key) {
            cout << "Found " << key << " at index " << i << endl;
            break;
        }
    }

    // Insert at end - O(1) if space available
    arr[n] = 15; n++; // arr = {7,12,9,11,3,15}

    // Delete from middle - O(n)
    int delIndex = 2;
    for (int i = delIndex; i < n - 1; i++)
        arr[i] = arr[i + 1];
    n--; // arr = {7,12,11,3,15}

    // Print array
    for (int i = 0; i < n; i++)
        cout << arr[i] << " ";
    return 0;
}
```

Bubble Sort

Bubble Sort: Bubble Sort is a simple comparison-based sorting algorithm that repeatedly steps through the list, compares adjacent elements, and swaps them if they are in the wrong order. The pass through the list is repeated until the list is sorted. The algorithm gets its name because larger elements "bubble" to the top (end) of the array with each pass.

How Bubble Sort Works

Bubble Sort works by comparing each pair of adjacent elements in the array and swapping them if they are in the wrong order. In each pass, the largest unsorted element moves to its correct position at the end. After the first pass, the largest element is at the last position. After the second pass, the second-largest is at the

second-to-last position, and so on. This continues until all elements are sorted.

Algorithm Steps

1. Start with the first element (index 0).
2. Compare the current element with the next element.
3. If current > next, swap them.
4. Move to the next pair and repeat until the end of the array.
5. After one pass, the largest element is at the end.
6. Repeat steps 1-5 for the remaining unsorted portion.
7. Continue until no swaps are needed in a complete pass.

Flowchart - Bubble Sort

```
START
|
v
Input: Array arr[], size n
|
v
Set i = 0
|
v
+-- i < n-1 ? ----- No -----> Print Sorted Array
| | |
| Yes v
| v STOP
| Set j = 0
| |
| v
| +-- j < n-i-1 ? -- No --> i++ ----> (back to i loop)
| | |
| | Yes
| | v
| | arr[j] > arr[j+1] ?
| | | |
| | Yes No
| | | |
| | v |
| | Swap |
| | arr[j],arr[j+1]
| | | |
| +---+-----+
| |
| v
| j++
+-----+
```

C++ Code: Bubble Sort

```
#include <iostream>
using namespace std;

void bubbleSort(int arr[], int n) {
// Outer loop: number of passes
for (int i = 0; i < n - 1; i++) {
```

```

// Inner loop: compare adjacent elements
// After i passes, last i elements are already sorted
for (int j = 0; j < n - i - 1; j++) {
    if (arr[j] > arr[j + 1]) {
        // Swap if current element is greater than next
        int temp = arr[j];
        arr[j] = arr[j + 1];
        arr[j + 1] = temp;
    }
}

int main() {
    int arr[] = {7, 12, 9, 11, 3};
    int n = sizeof(arr) / sizeof(arr[0]);
    bubbleSort(arr, n);
    cout << "Sorted: ";
    for (int i = 0; i < n; i++)
        cout << arr[i] << " ";
    return 0;
}

```

Dry Run: Bubble Sort

Input Array: [7, 12, 9, 11, 3], n = 5

Pass 1 (i=0):

Step	j	Compare	Action	Array State
1	0	7 vs 12	No swap (7 < 12)	[7, 12, 9, 11, 3]
2	1	12 vs 9	Swap (12 > 9)	[7, 9, 12, 11, 3]
3	2	12 vs 11	Swap (12 > 11)	[7, 9, 11, 12, 3]
4	3	12 vs 3	Swap (12 > 3)	[7, 9, 11, 3, 12]

After Pass 1: [7, 9, 11, 3, 12] - Largest element 12 is in place

Pass 2 (i=1):

Step	j	Compare	Action	Array State
1	0	7 vs 9	No swap	[7, 9, 11, 3, 12]
2	1	9 vs 11	No swap	[7, 9, 11, 3, 12]
3	2	11 vs 3	Swap	[7, 9, 3, 11, 12]

After Pass 2: [7, 9, 3, 11, 12] - 11 is in place

Pass 3 (i=2):

Step	j	Compare	Action	Array State
------	---	---------	--------	-------------

1	0	7 vs 9	No swap	[7, 9, 3, 11, 12]
2	1	9 vs 3	Swap	[7, 3, 9, 11, 12]

After Pass 3: [7, 3, 9, 11, 12] - 9 is in place

Pass 4 (i=3):

Step	j	Compare	Action	Array State
1	0	7 vs 3	Swap	[3, 7, 9, 11, 12]

After Pass 4: [3, 7, 9, 11, 12] - SORTED!

Passes and Steps Formula

Number of Passes: $n - 1$ (where n = number of elements)

Number of Comparisons (Steps): $n(n-1)/2 = n^2/2 - n/2$

For $n=5$: Passes = 4, Steps = 10 comparisons

Tip / Trick: Trick: The total number of comparisons in Bubble Sort follows the pattern: $(n-1) + (n-2) + \dots + 1 = n(n-1)/2$. This is the sum of first $(n-1)$ natural numbers.

Time Complexity - Bubble Sort

Case	Complexity	When
Best Case	$O(n)$	Array is already sorted (with optimized version using swap flag)
Average Case	$O(n^2)$	Array is randomly ordered
Worst Case	$O(n^2)$	Array is sorted in reverse order

Table 2.2: Bubble Sort Time Complexity

Space Complexity: $O(1)$ - Bubble Sort is an in-place algorithm. It only uses a constant amount of extra memory (one temporary variable for swapping).

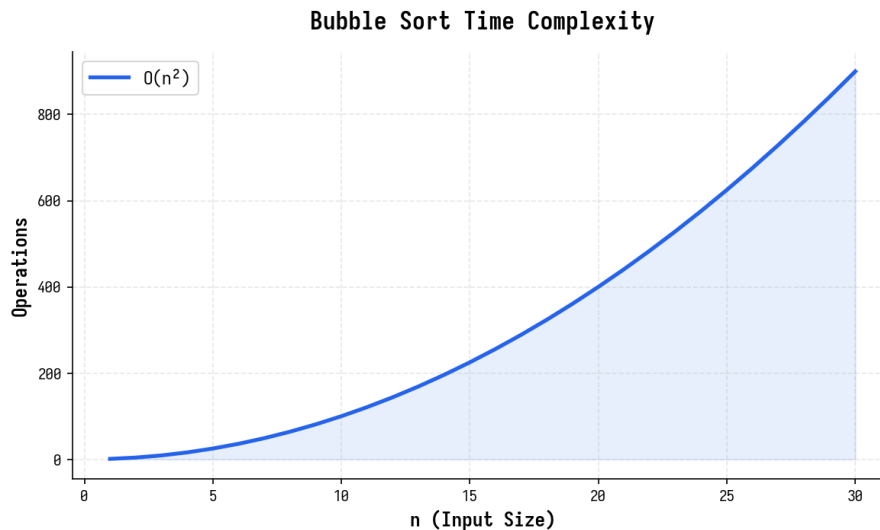


Figure 2.1: Bubble Sort Time Complexity Graph

Advantages & Disadvantages

Advantages:

- Simple to understand and implement.
- In-place sorting (requires no extra memory).
- Stable sort (preserves relative order of equal elements).
- Can be optimized to stop early if array is already sorted.

Disadvantages:

- Very slow for large datasets - $O(n^2)$ complexity.
- Too many unnecessary comparisons even if array is nearly sorted.
- Not suitable for production-level applications with large data.

Applications

- Educational purposes - teaching sorting concepts.
- Small datasets where simplicity matters more than efficiency.
- Checking if an array is already sorted (optimized version).

Important Points - Chapter 2

Key Takeaways:

1. Array = Fixed-size, contiguous, same-type elements with $O(1)$ access.
2. Bubble Sort: Compare adjacent, swap if needed, repeat. "Bubble up" largest element each pass.
3. Passes = $n - 1$; Comparisons = $n(n-1)/2$.

4. Time: Best $O(n)$ (optimized), Worst $O(n^2)$. Space: $O(1)$.
5. Bubble Sort is stable and in-place, but inefficient for large data.

Practice Questions - Chapter 2

Q1. What is the worst-case time complexity of Bubble Sort?

- (A) $O(n)$
- (B) $O(n \log n)$
- (C) $O(n^2)$
- (D) $O(\log n)$

Answer: C

Q2. How many passes does Bubble Sort need for an array of 6 elements?

- (A) 4
- (B) 5
- (C) 6
- (D) 7

Answer: B

Q3. After the first pass of Bubble Sort on [5,1,4,2,8], which element is in its final position?

- (A) 1
- (B) 2
- (C) 5
- (D) 8

Answer: D

Q4. Bubble Sort is which type of sort?

- (A) Stable
- (B) Unstable
- (C) Neither
- (D) Both

Answer: A

Q5. The space complexity of Bubble Sort is:

- (A) $O(n)$
- (B) $O(\log n)$
- (C) $O(1)$
- (D) $O(n^2)$

Answer: C

Chapter 3: Selection Sort

Selection Sort: Selection Sort is a comparison-based sorting algorithm that divides the array into a sorted and unsorted region. In each pass, it finds the minimum element from the unsorted region and places it at the beginning of the unsorted region, thus expanding the sorted region by one element.

The algorithm works by repeatedly selecting the smallest (or largest, for descending order) element from the unsorted portion of the array and swapping it with the first element of the unsorted portion. After each pass, one more element is added to the sorted portion at the beginning. Selection Sort is simple but inefficient for large lists, as it always makes $O(n^2)$ comparisons regardless of the initial order of the elements.

Algorithm Steps

1. Set the first element as the minimum.
2. Compare this minimum with all remaining unsorted elements.
3. If a smaller element is found, update the minimum.
4. After scanning all unsorted elements, swap the minimum with the first unsorted element.
5. Move the boundary of the sorted region one element to the right.
6. Repeat steps 1-5 until the entire array is sorted.

Flowchart - Selection Sort

```
START
|
v
Input: Array arr[], size n
|
v
Set i = 0
|
v
+-- i < n-1 ? ----- No -----> Print Sorted Array
| | |
| Yes v
| v STOP
| min_idx = i
| Set j = i + 1
| |
| v
| +-- j < n ? -- No --> Swap arr[i], arr[min_idx]
| | |
| | Yes v
| | v i++
| | arr[j] < arr[min_idx]? | |
| | | |
| | Yes No |
| | v | |
```



```

| | min_idx = j | |
| | | |
| +---+-----+ |
| | |
| v |
| j++ |
+-----+

```

C++ Code: Selection Sort

```

#include <iostream>
using namespace std;

void selectionSort(int arr[], int n) {
    // Outer loop: each pass places one element in correct position
    for (int i = 0; i < n - 1; i++) {
        int minIdx = i; // Assume first unsorted element is minimum
        // Inner loop: find the minimum in unsorted portion
        for (int j = i + 1; j < n; j++) {
            if (arr[j] < arr[minIdx])
                minIdx = j; // Update minimum index
        }
        // Swap minimum with first unsorted element
        if (minIdx != i) {
            int temp = arr[i];
            arr[i] = arr[minIdx];
            arr[minIdx] = temp;
        }
    }
}

int main() {
    int arr[] = {7, 12, 9, 11, 3};
    int n = sizeof(arr) / sizeof(arr[0]);
    selectionSort(arr, n);
    for (int i = 0; i < n; i++)
        cout << arr[i] << " ";
    return 0;
}

```

Dry Run: Selection Sort

Input Array: [7, 12, 9, 11, 3], n = 5

Pass 1 (i=0): Find minimum in entire array. Min = 3 at index 4. Swap arr[0] and arr[4].

Array: [7, 12, 9, 11, 3] -> [3, 12, 9, 11, 7]

Pass 2 (i=1): Find minimum from index 1 to end. Min = 7 at index 4. Swap arr[1] and arr[4].

Array: [3, 12, 9, 11, 7] -> [3, 7, 9, 11, 12]

Pass 3 (i=2): Find minimum from index 2 to end. Min = 9 at index 2 (already in place). No swap.

Array: [3, 7, 9, 11, 12] (unchanged)

Pass 4 (i=3): Find minimum from index 3 to end. Min = 11 at index 3 (already in place). No swap.

Array: [3, 7, 9, 11, 12] - SORTED!

Passes and Steps Formula

Passes = $n - 1$

Comparisons per pass: $(n-1) + (n-2) + \dots + 1 = n(n-1)/2$

Maximum Swaps: $n - 1$ (at most one swap per pass)

Tip / Trick: Selection Sort always makes $n(n-1)/2$ comparisons regardless of input, but at most $n-1$ swaps. This makes it useful when write operations (swaps) are expensive.

Time Complexity - Selection Sort

Case	Complexity	When
Best Case	$O(n^2)$	Even if sorted, all comparisons are still made
Average Case	$O(n^2)$	Randomly ordered array
Worst Case	$O(n^2)$	Reverse sorted array

Space Complexity: $O(1)$ - In-place algorithm. Only uses a constant amount of extra memory for the swap variable and index tracking.

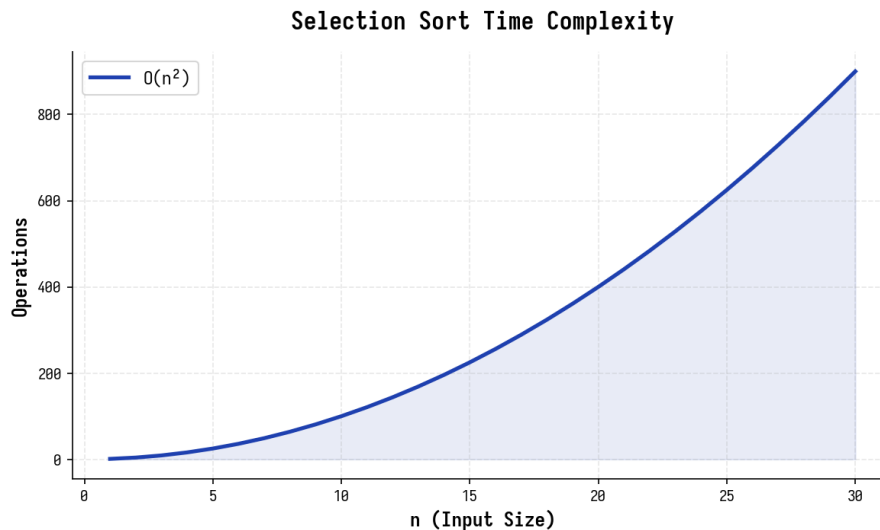


Figure 3.1: Selection Sort Time Complexity Graph

Advantages & Disadvantages

Advantages:

- Simple to understand and implement.
- In-place ($O(1)$ space).
- Minimizes the number of swaps (at most $n-1$). Good when swap cost is high.
- Performs well on small lists.

Disadvantages:

- Always $O(n^2)$ time even for nearly sorted arrays.
- Not stable (may change relative order of equal elements).
- Inefficient for large datasets.

Important Points - Chapter 3

Key Takeaways:

1. Selection Sort finds the minimum in each pass and places it at the correct position.
2. Passes = $n-1$; Comparisons = $n(n-1)/2$; Maximum Swaps = $n-1$.
3. Time: $O(n^2)$ for ALL cases. Space: $O(1)$.
4. NOT stable. In-place. Good when swap cost is high.
5. Unlike Bubble Sort, Selection Sort does NOT stop early for sorted input.

Practice Questions - Chapter 3

Q1. What is the best-case time complexity of Selection Sort?

- (A) $O(n)$
- (B) $O(n \log n)$
- (C) $O(n^2)$
- (D) $O(1)$

Answer: C

Q2. How many swaps does Selection Sort perform at most?

- (A) n
- (B) $n-1$
- (C) $n(n-1)/2$
- (D) n^2

Answer: B

Q3. Is Selection Sort stable?

- (A) Yes
- (B) No
- (C) Sometimes
- (D) Depends on input

Answer: B

Q4. In each pass, Selection Sort finds the:

- (A) Maximum element
- (B) Minimum element
- (C) Middle element
- (D) Random element

Answer: B

Q5. Selection Sort divides the array into:

- (A) Two equal halves
- (B) Sorted and unsorted regions
- (C) Odd and even elements
- (D) Positive and negative elements

Answer: B

Chapter 4: Insertion Sort

Insertion Sort: Insertion Sort is a simple sorting algorithm that works by iteratively inserting each element of an unsorted list into its correct position in the sorted portion of the list. It builds the sorted array one element at a time, similar to how you sort playing cards in your hand.

Insertion Sort is particularly efficient for small datasets and nearly sorted arrays. It is adaptive, meaning it runs faster when the input is already partially sorted. The algorithm maintains a sorted subarray at the beginning and expands it by inserting each new element in its correct position within the sorted portion. This process of finding the correct position involves shifting elements to the right to make room.

Algorithm Steps

1. Consider the first element as a sorted subarray (a single element is always sorted).
2. Take the next element (key) from the unsorted portion.
3. Compare the key with elements in the sorted portion from right to left.
4. Shift all sorted elements that are greater than the key one position to the right.
5. Insert the key in its correct position in the sorted portion.
6. Repeat steps 2-5 until all elements are sorted.

C++ Code: Insertion Sort

```
#include <iostream>
using namespace std;

void insertionSort(int arr[], int n) {
    for (int i = 1; i < n; i++) {
        int key = arr[i]; // Element to insert
        int j = i - 1;
        // Shift elements greater than key to the right
        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j--;
        }
        arr[j + 1] = key; // Insert key at correct position
    }
}

int main() {
    int arr[] = {7, 12, 9, 11, 3};
    int n = sizeof(arr) / sizeof(arr[0]);
    insertionSort(arr, n);
    for (int i = 0; i < n; i++)
        cout << arr[i] << " ";
    return 0;
}
```

Dry Run: Insertion Sort

Input Array: [7, 12, 9, 11, 3]

Pass 1 (i=1, key=12): 12 > 7, already in correct position. Array: [7, 12, 9, 11, 3]

Pass 2 (i=2, key=9): 9 < 12, shift 12 right. 9 > 7, insert at index 1. Array: [7, 9, 12, 11, 3]

Pass 3 (i=3, key=11): 11 < 12, shift 12 right. 11 > 9, insert at index 2. Array: [7, 9, 11, 12, 3]

Pass 4 (i=4, key=3): 3 < 12, shift right. 3 < 11, shift. 3 < 9, shift. 3 < 7, shift. Insert at index 0. Array: [3, 7, 9, 11, 12]

Final Sorted Array: [3, 7, 9, 11, 12]

Time Complexity - Insertion Sort

Case	Complexity	When
Best Case	$O(n)$	Array is already sorted (only n-1 comparisons, no shifts)
Average Case	$O(n^2)$	Array is randomly ordered
Worst Case	$O(n^2)$	Array is sorted in reverse order

Space Complexity: $O(1)$ - In-place algorithm. Only uses a constant amount of extra memory for the key variable and shift index.

Tip / Trick: Trick: Insertion Sort is ADAPTIVE - it runs much faster on nearly sorted data. If the array is already sorted, it only does n-1 comparisons and 0 shifts. This makes it ideal for online algorithms where data arrives one element at a time.

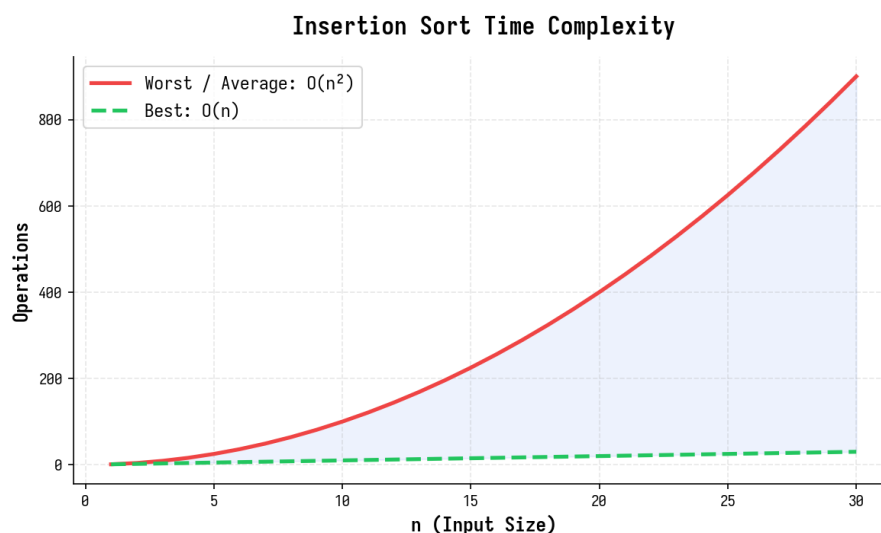


Figure 4.1: Insertion Sort Time Complexity

Advantages & Disadvantages

Advantages:

- Adaptive: Very fast for nearly sorted or small datasets.
- Stable: Preserves relative order of equal elements.
- In-place: $O(1)$ extra space.
- Online: Can sort as data arrives (no need for all data upfront).

Disadvantages:

- $O(n^2)$ worst and average case - slow for large datasets.
- Shifting elements is expensive for large arrays.

Important Points - Chapter 4

Key Takeaways:

1. Insertion Sort inserts each element into its correct position in the sorted portion.
2. Best Case: $O(n)$ when already sorted. Worst/Avg: $O(n^2)$. Space: $O(1)$.
3. Adaptive, stable, in-place, and online algorithm.
4. Like sorting playing cards in your hand.
5. Best choice for: small arrays, nearly sorted data, or streaming data.

Practice Questions - Chapter 4

Q1. Insertion Sort is which type of algorithm?

- (A) Non-adaptive
- (B) Adaptive
- (C) Neither
- (D) Both

Answer: B

Q2. What is the best-case time complexity of Insertion Sort?

- (A) $O(n^2)$
- (B) $O(n \log n)$
- (C) $O(n)$
- (D) $O(1)$

Answer: C

Q3. Is Insertion Sort stable?

- (A) Yes
- (B) No
- (C) Sometimes
- (D) Depends

Answer: A

Q4. Insertion Sort works like:

- (A) Sorting cards in hand
- (B) Finding minimum in array
- (C) Dividing array in half
- (D) Comparing adjacent elements

Answer: A

Q5. In the worst case, how many shifts does Insertion Sort perform?

- (A) n
- (B) $n-1$
- (C) $n(n-1)/2$
- (D) n^2

Answer: C

Chapter 5: Radix Sort

Radix Sort: Radix Sort is a non-comparison-based sorting algorithm that sorts elements by processing them digit by digit, from the least significant digit (LSD) to the most significant digit (MSD). It uses Counting Sort as a subroutine to sort digits at each position.

Unlike comparison-based sorts (Bubble, Selection, Insertion, Merge, Quick, Heap), Radix Sort does not compare elements directly. Instead, it distributes elements into buckets based on individual digits, starting from the least significant digit. This allows it to achieve linear time complexity for fixed-length keys. Radix Sort is particularly efficient for sorting integers and strings with fixed-length keys.

Algorithm Steps

1. Find the maximum element in the array to determine the number of digits (k).
2. For each digit position from LSD to MSD (ones, tens, hundreds...):
 - a. Distribute elements into 10 buckets (0-9) based on the current digit.
 - b. Collect elements from buckets in order (0 to 9).
3. After processing all digit positions, the array is sorted.

C++ Code: Radix Sort


```

#include <iostream>
using namespace std;

// Get the maximum value to determine number of digits
int getMax(int arr[], int n) {
    int maxVal = arr[0];
    for (int i = 1; i < n; i++)
        if (arr[i] > maxVal) maxVal = arr[i];
    return maxVal;
}

// Counting Sort for a specific digit (exp = 1, 10, 100...)
void countingSort(int arr[], int n, int exp) {
    int output[100]; // Output array
    int count[10] = {0}; // Count array for digits 0-9

    // Count occurrences of each digit
    for (int i = 0; i < n; i++)
        count[(arr[i] / exp) % 10]++;

    // Change count to cumulative positions
    for (int i = 1; i < 10; i++)
        count[i] += count[i - 1];

    // Build output array (reverse for stability)
    for (int i = n - 1; i >= 0; i--) {
        int digit = (arr[i] / exp) % 10;
        output[count[digit] - 1] = arr[i];
        count[digit]--;
    }

    // Copy output to original array
    for (int i = 0; i < n; i++)
        arr[i] = output[i];
}

void radixSort(int arr[], int n) {
    int maxVal = getMax(arr, n);
    // Sort by each digit position
    for (int exp = 1; maxVal / exp > 0; exp *= 10)
        countingSort(arr, n, exp);
}

int main() {
    int arr[] = {170, 45, 75, 90, 802, 24, 2, 66};
    int n = sizeof(arr) / sizeof(arr[0]);
    radixSort(arr, n);
    for (int i = 0; i < n; i++)
        cout << arr[i] << " ";
    return 0;
}

```

Dry Run: Radix Sort

Input Array: [170, 45, 75, 90, 802, 24, 2, 66]

Maximum = 802, so $k = 3$ digits. We need 3 passes.

Pass 1 - Sort by Unit Digit (exp=1):

Bucket distribution: 0:[170,90], 2:[802,2], 4:[24], 5:[45,75], 6:[66]

After collecting: [170, 90, 802, 2, 24, 45, 75, 66]

Pass 2 - Sort by Tens Digit (exp=10):

Bucket distribution: 0:[802,2], 2:[24], 4:[45], 6:[66], 7:[170,75], 9:[90]

After collecting: [802, 2, 24, 45, 66, 170, 75, 90]

Pass 3 - Sort by Hundreds Digit (exp=100):

Bucket distribution: 0:[2,24,45,66,75,90], 1:[170], 8:[802]

After collecting: [2, 24, 45, 66, 75, 90, 170, 802] - SORTED!

Time Complexity

Case	Complexity	When
Best Case	$O(n \times k)$	k = number of digits in the largest number
Average Case	$O(n \times k)$	Depends on n and k
Worst Case	$O(n \times k)$	k can be $\log(\max_value)$, making it $O(n \log(\max))$

Space Complexity: $O(n + k)$ where n is the number of elements and k is the range of digits (10 for decimal). Extra space is needed for the count array and output array.

Tip / Trick: Radix Sort is NOT a comparison sort. It sorts by digits, so it can beat $O(n \log n)$ for fixed-length keys. However, if k (number of digits) approaches n , the complexity becomes $O(n^2)$.

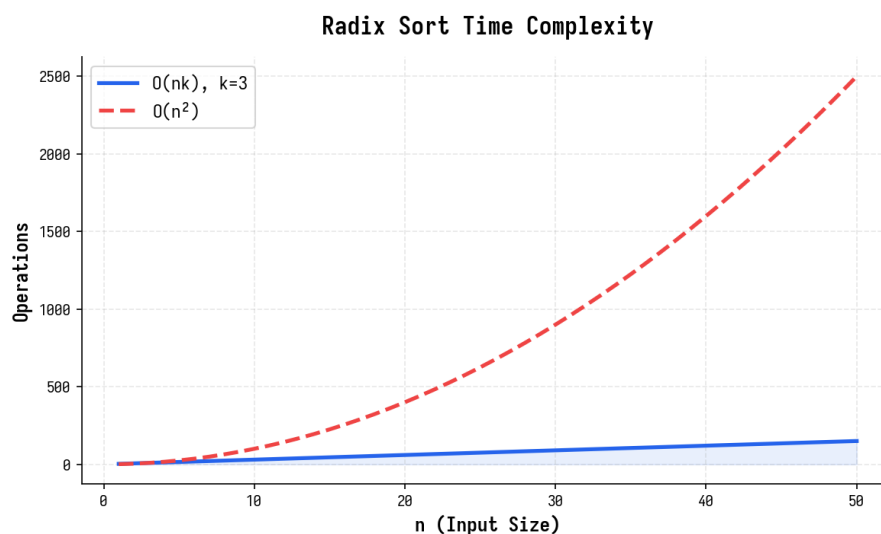


Figure 5.1: Radix Sort Time Complexity

Advantages & Disadvantages

Advantages:

- Linear time for fixed-length keys - can be faster than $O(n \log n)$ sorts.
- Stable sort.
- Non-comparison based - works differently from most sorting algorithms.

Disadvantages:

- Only works for integers and strings (not for floating-point or complex types).
- Requires extra space $O(n + k)$.
- Performance depends on the number of digits (k).

Important Points - Chapter 5

Key Takeaways:

1. Radix Sort is non-comparison based; sorts by individual digits (LSD to MSD).
2. Time: $O(n \times k)$. Space: $O(n + k)$.
3. Uses Counting Sort as subroutine for each digit position.
4. Stable, but NOT in-place. Works only on integers/strings.
5. k = number of digits in the maximum element.

Practice Questions - Chapter 5

Q1. Radix Sort is what type of sorting algorithm?

- (A) Comparison-based
- (B) Non-comparison based
- (C) Hybrid
- (D) None

Answer: B

Q2. What is the time complexity of Radix Sort?

- (A) $O(n \log n)$
- (B) $O(n \times k)$
- (C) $O(n^2)$
- (D) $O(k)$

Answer: B

Q3. Radix Sort processes digits from:

- (A) MSD to LSD
- (B) LSD to MSD
- (C) Random order
- (D) Middle digit outward

Answer: B

Q4. Radix Sort uses which algorithm as a subroutine?

- (A) Quick Sort
- (B) Merge Sort
- (C) Counting Sort
- (D) Bubble Sort

Answer: C

Q5. Radix Sort is stable?

- (A) Yes
- (B) No
- (C) Sometimes
- (D) Never

Answer: A

Chapter 6: Linear Search

Linear Search: Linear Search (also called Sequential Search) is the simplest searching algorithm. It iterates over all elements of the array one by one and checks if the current element is equal to the target element. If found, it returns the index; otherwise, it returns -1.

Linear Search is straightforward and does not require the data to be sorted. It works on any data structure that can be traversed sequentially. While it is simple and works in all cases, it becomes very slow for large datasets because it may need to check every single element.

Algorithm Steps

1. Start from the first element (index 0).
2. Compare the current element with the target (key).
3. If they match, return the current index (found!).
4. If they do not match, move to the next element.
5. Repeat steps 2-4 until the end of the array.
6. If the end is reached without finding the key, return -1 (not found).

C++ Code: Linear Search

```
#include <iostream>
using namespace std;

int linearSearch(int arr[], int n, int key) {
    // Iterate through each element
    for (int i = 0; i < n; i++) {
        if (arr[i] == key)
            return i; // Key found at index i
    }
    return -1; // Key not found
}

int main() {
    int arr[] = {10, 50, 30, 70, 80, 20, 90, 40};
    int n = sizeof(arr) / sizeof(arr[0]);
    int key = 30;
    int result = linearSearch(arr, n, key);
    if (result != -1)
        cout << "Found at index " << result;
    else
        cout << "Not found";
    return 0;
}
```

Dry Run: Linear Search

Array: [10, 50, 30, 70, 80, 20, 90, 40], Key = 30

Step	Index	Value	Key Match?	Action
1	0	10	No	Move to next
2	1	50	No	Move to next
3	2	30	Yes!	Return index 2

Result: Key 30 found at index 2 (3 comparisons)

Time Complexity - Linear Search

Case	Complexity	When
Best Case	O(1)	Key is at the first position (1 comparison)
Average Case	O(n)	Key is somewhere in the middle
Worst Case	O(n)	Key is at the last position or not present

Space Complexity: O(1) - No extra memory is needed beyond the input array and a few variables.

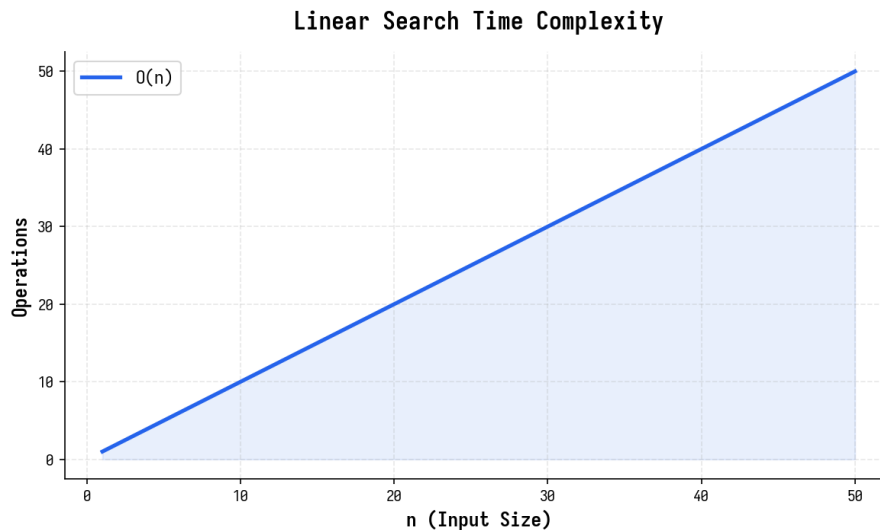


Figure 6.1: Linear Search Time Complexity

Advantages & Disadvantages

Advantages:

- Simple to implement and understand.
- Works on both sorted and unsorted arrays.
- Works on any data structure that can be traversed.
- No extra memory required.

Disadvantages:

- Very slow for large datasets - $O(n)$ in the worst case.
- Inefficient compared to Binary Search for sorted data.

Important Points - Chapter 6

Key Takeaways:

1. Linear Search = Check each element one by one (sequential).
2. Best: $O(1)$, Worst/Avg: $O(n)$. Space: $O(1)$.
3. Works on sorted AND unsorted data.
4. Also called Sequential Search.
5. Use when data is unsorted or dataset is small.

Practice Questions - Chapter 6

Q1. What is the best-case time complexity of Linear Search?

- (A) $O(n)$
- (B) $O(1)$

(C) $O(\log n)$

(D) $O(n^2)$

Answer: B

Q2. Linear Search is also known as:

(A) Binary Search

(B) Sequential Search

(C) Jump Search

(D) Hash Search

Answer: B

Q3. Linear Search requires the array to be:

(A) Sorted

(B) Unsorted

(C) Either sorted or unsorted

(D) Unique elements only

Answer: C

Q4. In the worst case, Linear Search makes how many comparisons?

(A) n

(B) $n/2$

(C) $n-1$

(D) $2n$

Answer: A

Q5. If key is not found, Linear Search returns:

(A) 0

(B) 1

(C) -1

(D) n

Answer: C

Chapter 7: Binary Search

Binary Search: Binary Search is a searching algorithm that operates on a sorted array by repeatedly dividing the search space into two halves. It compares the target value with the middle element and eliminates half of the search space in each step, achieving $O(\log n)$ time complexity.

Binary Search is dramatically faster than Linear Search for large sorted datasets. Instead of checking every element, it eliminates half of the remaining elements in

each step. This is similar to looking up a word in a dictionary - you open to the middle, check if the word is before or after that page, and repeat with the appropriate half. The key requirement is that the data **MUST** be sorted before applying Binary Search.

Binary Search Conditions

- The data structure **MUST** be sorted.
- Access to any element should take constant time ($O(1)$).
- The search space must be monotonic (consistently increasing or decreasing).

Algorithm Steps

1. Set $low = 0$, $high = n - 1$.
2. While $low \leq high$:
 - a. Calculate $mid = low + (high - low) / 2$.
 - b. If $arr[mid] == key$, return mid (found!).
 - c. If $arr[mid] < key$, search the right half: $low = mid + 1$.
 - d. If $arr[mid] > key$, search the left half: $high = mid - 1$.
3. If $low > high$, return -1 (not found).

Note: Why $mid = low + (high - low) / 2$ instead of $mid = (low + high) / 2$? Because $(low + high)$ can overflow for very large values. The formula $low + (high - low) / 2$ avoids this overflow.

C++ Code: Binary Search

```
#include <iostream>
using namespace std;

int binarySearch(int arr[], int n, int key) {
    int low = 0, high = n - 1;
    while (low <= high) {
        int mid = low + (high - low) / 2; // Avoid overflow
        if (arr[mid] == key)
            return mid; // Key found
        else if (arr[mid] < key)
            low = mid + 1; // Search right half
        else
            high = mid - 1; // Search left half
    }
    return -1; // Key not found
}

int main() {
    int arr[] = {2, 5, 8, 12, 16, 23, 38, 56, 72, 91};
    int n = sizeof(arr) / sizeof(arr[0]);
    int key = 23;
```



```

int result = binarySearch(arr, n, key);
if (result != -1)
cout << "Found at index " << result;
else
cout << "Not found";
return 0;
}

```

Dry Run: Binary Search

Array: [2, 5, 8, 12, 16, 23, 38, 56, 72, 91], Key = 23

Step	Low	High	Mid	arr[Mid]	Comparison	Action
1	0	9	4	16	23 > 16	Search right: low = 5
2	5	9	7	56	23 < 56	Search left: high = 6
3	5	6	5	23	23 == 23	FOUND at index 5!

Result: Key 23 found at index 5 in only 3 steps!

Tip / Trick: Trick: Binary Search on 1 million elements takes at most 20 comparisons ($\log_2(1,000,000) = 19.93$). Compare this to Linear Search which could take up to 1,000,000 comparisons!

Time Complexity - Binary Search

Case	Complexity	When
Best Case	O(1)	Key is at the middle position on first check
Average Case	O(log n)	Key is somewhere in the array
Worst Case	O(log n)	Key is at the edge or not present

Space Complexity: O(1) for iterative version (only a few variables). O(log n) for recursive version (due to call stack).

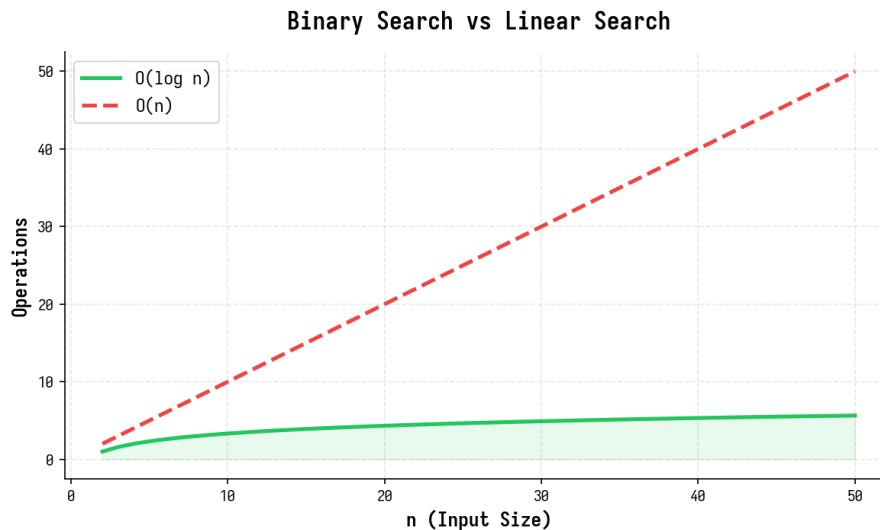


Figure 7.1: Binary Search vs Linear Search Complexity

Advantages & Disadvantages

Advantages:

- Extremely fast - $O(\log n)$ for large sorted datasets.
- In-place for iterative version - $O(1)$ space.
- Simple to implement.

Disadvantages:

- REQUIRES sorted data. If unsorted, must sort first ($O(n \log n)$).
- Not suitable for data structures without random access (like linked lists).
- Not good for small datasets (overhead of dividing is not worth it).

Important Points - Chapter 7

Key Takeaways:

1. Binary Search requires SORTED data. Eliminates half the search space each step.
2. Best: $O(1)$, Worst/Avg: $O(\log n)$. Space: $O(1)$ iterative, $O(\log n)$ recursive.
3. Use $\text{mid} = \text{low} + (\text{high} - \text{low}) / 2$ to avoid integer overflow.
4. Like looking up a word in a dictionary - check middle, go left or right.
5. For 1 million elements: Binary Search = ~20 steps, Linear Search = up to 1,000,000 steps.

Practice Questions - Chapter 7

Q1. Binary Search requires the array to be:

- (A) Sorted
- (B) Unsorted
- (C) Either
- (D) Reverse sorted

Answer: A

Q2. What is the worst-case time complexity of Binary Search?

- (A) $O(n)$
- (B) $O(\log n)$
- (C) $O(n \log n)$
- (D) $O(1)$

Answer: B

Q3. Why use $\text{mid} = \text{low} + (\text{high} - \text{low}) / 2$ instead of $(\text{low} + \text{high}) / 2$?

- (A) Faster
- (B) Avoids overflow
- (C) More accurate
- (D) Better style

Answer: B

Q4. How many comparisons to search 1 million elements?

- (A) 1,000,000
- (B) 1000
- (C) 20
- (D) 500

Answer: C

Q5. Binary Search is NOT suitable for:

- (A) Sorted arrays
- (B) Large sorted datasets
- (C) Linked lists
- (D) Integer arrays

Answer: C

Chapter 8: Merge Sort

Merge Sort: Merge Sort is a divide-and-conquer sorting algorithm that works by recursively dividing the array into two halves, sorting each half, and then merging the sorted halves back together. It guarantees $O(n \log n)$ time complexity in all cases.

Merge Sort is one of the most efficient and reliable sorting algorithms. Unlike Quick Sort, which can degrade to $O(n^2)$ in the worst case, Merge Sort always runs in $O(n \log n)$ time. The trade-off is that it requires $O(n)$ extra space for the temporary arrays used during merging. Merge Sort is the algorithm of choice when stability is required and when the dataset is too large to fit in memory (external sorting).

How Merge Sort Works

Merge Sort follows the Divide and Conquer paradigm with three steps:

- Divide: Split the array into two halves recursively until each subarray has only one element (which is trivially sorted).
- Conquer: Each subarray of one element is already sorted.
- Merge: Merge the sorted subarrays back together in sorted order. The merging process compares elements from both halves and places them in the correct order.

C++ Code: Merge Sort

```
#include <iostream>
using namespace std;

// Merge two sorted subarrays
void merge(int arr[], int left, int mid, int right) {
    int n1 = mid - left + 1; // Size of left subarray
    int n2 = right - mid; // Size of right subarray

    // Create temporary arrays
    int L[50], R[50];
    for (int i = 0; i < n1; i++) L[i] = arr[left + i];
    for (int j = 0; j < n2; j++) R[j] = arr[mid + 1 + j];

    // Merge the temp arrays back into arr[]
    int i = 0, j = 0, k = left;
    while (i < n1 && j < n2) {
        if (L[i] <= R[j]) {
            arr[k] = L[i]; i++;
        } else {
            arr[k] = R[j]; j++;
        }
        k++;
    }
    // Copy remaining elements of L[]
    while (i < n1) { arr[k] = L[i]; i++; k++; }
    // Copy remaining elements of R[]
    while (j < n2) { arr[k] = R[j]; j++; k++; }
}

// Recursive Merge Sort
void mergeSort(int arr[], int left, int right) {
```

```

if (left < right) {
    int mid = left + (right - left) / 2;
    mergeSort(arr, left, mid); // Sort left half
    mergeSort(arr, mid + 1, right); // Sort right half
    merge(arr, left, mid, right); // Merge sorted halves
}
}

int main() {
    int arr[] = {38, 27, 43, 10};
    int n = sizeof(arr) / sizeof(arr[0]);
    mergeSort(arr, 0, n - 1);
    for (int i = 0; i < n; i++)
        cout << arr[i] << " ";
    return 0;
}

```

Dry Run: Merge Sort

Input Array: [38, 27, 43, 10]

Step 1 - Divide: Split [38, 27, 43, 10] into [38, 27] and [43, 10]

Step 2 - Divide: Split [38, 27] into [38] and [27]. Split [43, 10] into [43] and [10]

Step 3 - Merge: Merge [38] and [27] into [27, 38]. Merge [43] and [10] into [10, 43]

Step 4 - Merge: Merge [27, 38] and [10, 43] into [10, 27, 38, 43]

Detailed Merge of Step 4:

Step	L[]	R[]	Compare	Pick	Result
1	27,38	10,43	27 vs 10	10	[10]
2	27,38	43	27 vs 43	27	[10,27]
3	38	43	38 vs 43	38	[10,27,38]
4	-	43	Only R left	43	[10,27,38,43]

Final Sorted Array: [10, 27, 38, 43]

Time Complexity - Merge Sort

Case	Complexity	When
Best Case	$O(n \log n)$	Already sorted or nearly sorted
Average Case	$O(n \log n)$	Randomly ordered array
Worst Case	$O(n \log n)$	Reverse sorted array

Space Complexity: $O(n)$ - Merge Sort requires additional memory proportional to the size of the input array for the temporary left and right subarrays used during merging.

Tip / Trick: Merge Sort always takes $O(n \log n)$ regardless of input. The recursion tree has $\log n$ levels, and at each level, merging takes $O(n)$ time. Total = $n \times \log n = O(n \log n)$.

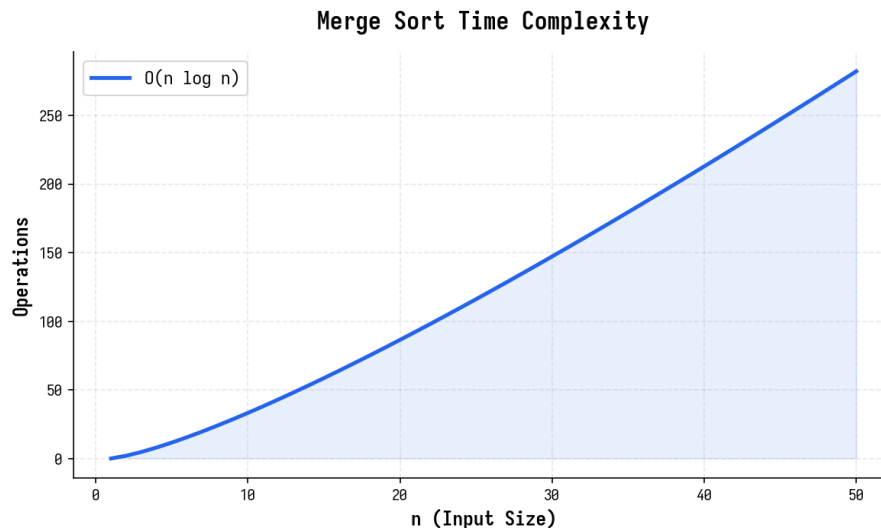


Figure 8.1: Merge Sort Time Complexity

Advantages & Disadvantages

Advantages:

- Guaranteed $O(n \log n)$ in ALL cases - no worst-case surprises.
- Stable sort - preserves relative order of equal elements.
- Suitable for external sorting (data too large for memory).
- Works well with linked lists (no random access needed).

Disadvantages:

- Requires $O(n)$ extra space - NOT in-place.
- Slower in practice than Quick Sort for small arrays due to overhead.
- Recursive calls use stack space.

Important Points - Chapter 8

Key Takeaways:

1. Merge Sort = Divide + Conquer + Merge. Always $O(n \log n)$.
2. Space: $O(n)$ - NOT in-place. Stable.
3. The recursion tree has $\log n$ levels; merging at each level costs $O(n)$.

4. Best choice for: linked lists, external sorting, when stability matters.
5. Merge Sort is the ONLY stable $O(n \log n)$ sort in this course.

Practice Questions - Chapter 8

Q1. Merge Sort uses which algorithm paradigm?

- (A) Greedy
- (B) Dynamic Programming
- (C) Divide and Conquer
- (D) Backtracking

Answer: C

Q2. What is the space complexity of Merge Sort?

- (A) $O(1)$
- (B) $O(\log n)$
- (C) $O(n)$
- (D) $O(n \log n)$

Answer: C

Q3. Merge Sort is stable?

- (A) Yes
- (B) No
- (C) Sometimes
- (D) Depends on implementation

Answer: A

Q4. The worst-case time complexity of Merge Sort is:

- (A) $O(n)$
- (B) $O(n \log n)$
- (C) $O(n^2)$
- (D) $O(\log n)$

Answer: B

Q5. Merge Sort is best suited for:

- (A) Small arrays
- (B) In-place sorting
- (C) External sorting
- (D) Integer-only sorting

Answer: C

Chapter 9: Quick Sort

Quick Sort: Quick Sort is a divide-and-conquer sorting algorithm that selects a pivot element and partitions the array such that all elements smaller than the pivot are on the left and all elements larger are on the right. It then recursively sorts the left and right partitions.

Quick Sort is one of the fastest sorting algorithms in practice. It is the basis for many standard library sort functions (C++ `std::sort`, Java `Arrays.sort` for primitives). The key to Quick Sort performance is the choice of pivot. A good pivot divides the array roughly in half, leading to $O(n \log n)$ performance. A bad pivot (always the smallest or largest element) leads to $O(n^2)$ worst-case performance.

Choice of Pivot Element

- First or Last Element: Simple to implement, but leads to worst case $O(n^2)$ if array is already sorted.
- Random Element: Preferred because it avoids predictable patterns that trigger the worst case.
- Median Element: Ideal for time complexity as it consistently divides the array in half, though finding the median has its own cost.
- Median of Three: Choose the median of first, middle, and last elements. Good practical choice.

C++ Code: Quick Sort

```
#include <iostream>
using namespace std;

// Swap two elements
void swap(int& a, int& b) {
    int temp = a; a = b; b = temp;
}

// Partition function (Lomuto scheme)
int partition(int arr[], int low, int high) {
    int pivot = arr[high]; // Last element as pivot
    int i = low - 1; // Index of smaller element
    for (int j = low; j <= high - 1; j++) {
        if (arr[j] < pivot) {
            i++;
            swap(arr[i], arr[j]);
        }
    }
    swap(arr[i + 1], arr[high]); // Place pivot in correct position
    return i + 1; // Return pivot index
}
```



```
// Quick Sort recursive function
void quickSort(int arr[], int low, int high) {
    if (low < high) {
        int pi = partition(arr, low, high); // Partition index
        quickSort(arr, low, pi - 1); // Sort left of pivot
        quickSort(arr, pi + 1, high); // Sort right of pivot
    }
}

int main() {
    int arr[] = {11, 9, 12, 7, 3};
    int n = sizeof(arr) / sizeof(arr[0]);
    quickSort(arr, 0, n - 1);
    for (int i = 0; i < n; i++)
        cout << arr[i] << " ";
    return 0;
}
```

Dry Run: Quick Sort

Input: [11, 9, 12, 7, 3], Pivot = last element

Pass 1: Pivot = 3. All elements > 3, so swap 3 with 11. Result: [3, 9, 12, 7, 11].
Pivot 3 is at index 0.

Pass 2: Sort [9, 12, 7, 11]. Pivot = 11. 9 and 7 go left, 12 goes right. Swap 11 with 12. Result: [3, 9, 7, 11, 12].

Pass 3: Sort [9, 7]. Pivot = 7. 7 < 9, so swap. Result: [3, 7, 9, 11, 12].

Final: [3, 7, 9, 11, 12] - SORTED!

Time Complexity - Quick Sort

Case	Complexity	When
Best Case	$O(n \log n)$	Pivot always divides array into two equal halves
Average Case	$O(n \log n)$	Pivot divides reasonably well
Worst Case	$O(n^2)$	Pivot is always smallest or largest element

Space Complexity: $O(\log n)$ average for the recursive call stack. $O(n)$ worst case when the recursion is skewed.

Tip / Trick: Trick: To avoid worst case in Quick Sort, always use Randomized Pivot or Median-of-Three. This makes worst case extremely unlikely in practice.

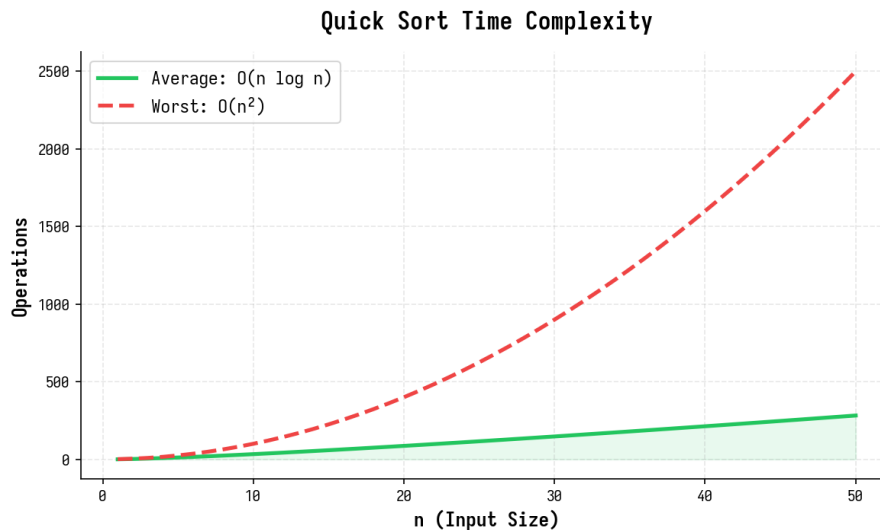


Figure 9.1: Quick Sort Time Complexity

Advantages & Disadvantages

Advantages:

- Fastest in practice on average for most datasets.
- In-place - $O(\log n)$ extra space (stack).
- Cache-friendly - works on original array.
- Used in standard libraries (C++ `std::sort`, Java `Arrays.sort`).

Disadvantages:

- Worst case $O(n^2)$ with bad pivot choice.
- NOT stable - does not preserve order of equal elements.
- Recursive (stack overhead).

Important Points - Chapter 9

Key Takeaways:

1. Quick Sort: Pick pivot, partition (smaller left, larger right), recurse.
2. Best/Avg: $O(n \log n)$. Worst: $O(n^2)$ with bad pivot.
3. Space: $O(\log n)$ average. NOT stable. In-place.
4. Use random pivot or median-of-three to avoid worst case.
5. Fastest practical sort. Used in C++ `std::sort` and Java `Arrays.sort`.

Practice Questions - Chapter 9

Q1. The worst-case time complexity of Quick Sort is:

- (A) $O(n \log n)$

- (B) $O(n)$
- (C) $O(n^2)$
- (D) $O(\log n)$

Answer: C

Q2. Quick Sort is a:

- (A) Stable sort
- (B) Unstable sort
- (C) Both
- (D) Neither

Answer: B

Q3. Which pivot selection avoids worst case?

- (A) First element
- (B) Last element
- (C) Random element
- (D) Second element

Answer: C

Q4. Quick Sort partition function places the pivot in its:

- (A) First position
- (B) Last position
- (C) Correct sorted position
- (D) Random position

Answer: C

Q5. The space complexity of Quick Sort (average) is:

- (A) $O(n)$
- (B) $O(1)$
- (C) $O(\log n)$
- (D) $O(n \log n)$

Answer: C

Chapter 10: Heap Sort

Heap Sort: Heap Sort is a comparison-based sorting algorithm that uses a Binary Heap data structure. It first builds a Max Heap from the array, then repeatedly extracts the maximum element (root) and places it at the end, reducing the heap size each time until the array is sorted.

Heap Sort is an optimized version of Selection Sort. Instead of scanning the entire unsorted portion to find the maximum ($O(n)$), it uses a Max Heap to access the maximum in $O(1)$ and maintain the heap property in $O(\log n)$. The result is a guaranteed $O(n \log n)$ algorithm that works in-place. However, it is not stable and has higher constant factors than Merge Sort in practice.

What is a Binary Heap?

A Binary Heap is a complete binary tree where every parent node is greater than or equal to its children (Max Heap) or less than or equal to its children (Min Heap). It is typically stored as an array where for a node at index i : left child is at $2i+1$, right child is at $2i+2$, and parent is at $(i-1)/2$.

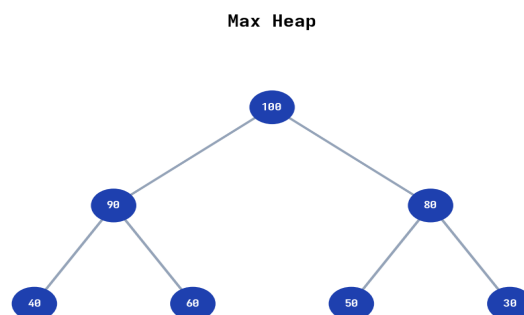


Figure 10.1: Max Heap Tree Structure

Working Process

1. Build Max Heap: Convert the array into a max heap using heapify (bottom-up).
2. Extract Max: Swap the root (maximum) with the last element.
3. Heapify: Restore the max heap property on the reduced heap.
4. Repeat: Continue extracting the max and heapifying until only one element remains.

C++ Code: Heap Sort

```
#include <iostream>
using namespace std;

// Heapify a subtree rooted at index i
void heapify(int arr[], int n, int i) {
    int largest = i; // Initialize largest as root
    int left = 2 * i + 1; // Left child index
    int right = 2 * i + 2; // Right child index

    // If left child is larger than root
    if (left < n && arr[left] > arr[largest])
```

```

largest = left;
// If right child is larger than current largest
if (right < n && arr[right] > arr[largest])
largest = right;

// If largest is not root, swap and continue heapifying
if (largest != i) {
int temp = arr[i];
arr[i] = arr[largest];
arr[largest] = temp;
heapify(arr, n, largest); // Recursively heapify
}
}

void heapSort(int arr[], int n) {
// Step 1: Build Max Heap (start from last non-leaf node)
for (int i = n / 2 - 1; i >= 0; i--)
heapify(arr, n, i);

// Step 2: Extract elements one by one
for (int i = n - 1; i > 0; i--) {
// Move current root to end
int temp = arr[0];
arr[0] = arr[i];
arr[i] = temp;
// Heapify the reduced heap
heapify(arr, i, 0);
}
}

int main() {
int arr[] = {9, 4, 3, 8, 10, 2, 5};
int n = sizeof(arr) / sizeof(arr[0]);
heapSort(arr, n);
for (int i = 0; i < n; i++)
cout << arr[i] << " ";
return 0;
}

```

Dry Run: Heap Sort

Input Array: [9, 4, 3, 8, 10, 2, 5]

Step 1: Build Max Heap

Start from last non-leaf node (index 2, value 3). Compare 3 with children 2 and 5. $5 > 3$, so swap. Array: [9, 4, 5, 8, 10, 2, 3]

Move to index 1 (value 4). Compare 4 with children 8 and 10. $10 > 4$, swap. Array: [9, 10, 5, 8, 4, 2, 3]

Move to index 0 (value 9). Compare 9 with children 10 and 5. $10 > 9$, swap. Array: [10, 9, 5, 8, 4, 2, 3] - Max Heap built!

Step 2: Extract and Sort

Swap root(10) with last(3), heapify: [9, 8, 5, 3, 4, 2, |10]

Swap root(9) with last(2), heapify: [8, 4, 5, 3, 2, |9, 10]

Swap root(8) with last(2), heapify: [5, 4, 2, 3, |8, 9, 10]

Continue until sorted: [2, 3, 4, 5, 8, 9, 10]

Time Complexity - Heap Sort

Case	Complexity	When
Best Case	$O(n \log n)$	All elements are the same (no swaps needed in heapify)
Average Case	$O(n \log n)$	Randomly ordered array
Worst Case	$O(n \log n)$	Reverse sorted array

Space Complexity: $O(1)$ - Heap Sort is in-place. It uses only a constant amount of extra memory for variables. The recursive heapify can be made iterative for $O(1)$ stack space.

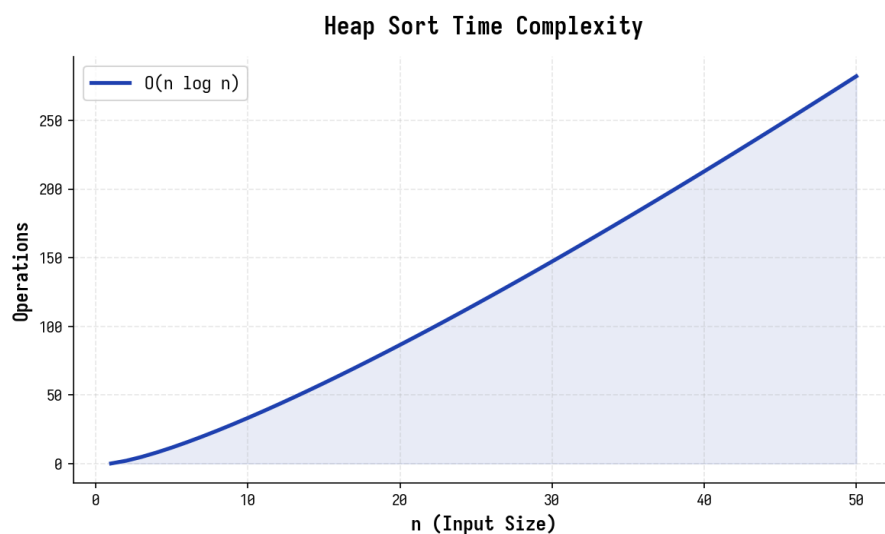


Figure 10.2: Heap Sort Time Complexity

Advantages & Disadvantages

Advantages:

- Guaranteed $O(n \log n)$ in ALL cases.
- In-place - $O(1)$ extra space.
- Simple to understand with the heap structure.

Disadvantages:

- Not stable - may rearrange equal elements.

- Higher constant factors - slower than Quick Sort in practice.
- Poor cache performance due to non-sequential memory access.

Important Points - Chapter 10

Key Takeaways:

1. Heap Sort uses Max Heap: Build heap, extract max, heapify, repeat.
2. Time: $O(n \log n)$ in ALL cases. Space: $O(1)$. In-place.
3. NOT stable. Slower in practice than Quick Sort.
4. Heap property: parent $>$ children (Max Heap).
5. Array to tree: left = $2i+1$, right = $2i+2$, parent = $(i-1)/2$.

Practice Questions - Chapter 10

Q1. Heap Sort is based on which data structure?

- (A) Stack
- (B) Queue
- (C) Binary Heap
- (D) Graph

Answer: C

Q2. The worst-case time complexity of Heap Sort is:

- (A) $O(n)$
- (B) $O(n \log n)$
- (C) $O(n^2)$
- (D) $O(\log n)$

Answer: B

Q3. Is Heap Sort stable?

- (A) Yes
- (B) No
- (C) Sometimes
- (D) Always

Answer: B

Q4. In a Max Heap, the root element is the:

- (A) Minimum
- (B) Maximum
- (C) Median
- (D) Average

Answer: B

Q5. The left child of node at index i is at:

- (A) $2i$
- (B) $2i+1$
- (C) $2i+2$
- (D) $i/2$

Answer: B

Chapter 11: Linear Data Structures

Linear Data Structure: A Linear Data Structure is a type of data structure where elements are arranged sequentially or linearly. Each element has a previous and next adjacent neighbor, except for the first and last elements. The elements are traversed in a single run.

Characteristics:

- **Sequential Organization:** Elements are arranged one after another.
- **Order Preservation:** The addition order is maintained.
- **Fixed or Dynamic Size:** Arrays are fixed; linked lists, stacks, and queues can be dynamic.
- **Efficient Access:** Some offer direct access like arrays ($O(1)$).

1. Array

(Already covered in Chapter 2 - Arrays store same-type elements at contiguous locations with $O(1)$ random access.)

2. Linked List

Linked List: A Linked List is a linear data structure where elements (nodes) are connected using pointers. Each node contains a data field and a reference (link) to the next node. Unlike arrays, elements are NOT stored at contiguous memory locations.

Singly Linked List

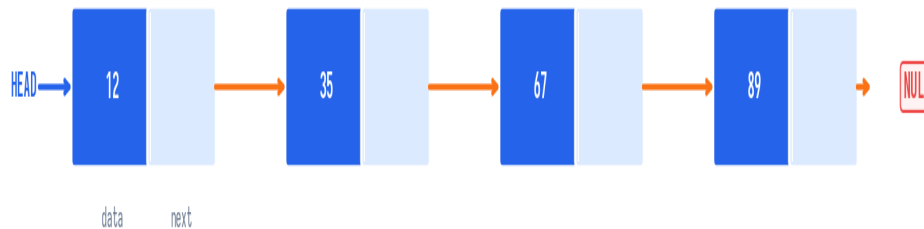


Figure 11.1: Singly Linked List Structure

Types of Linked Lists

- Singly Linked List: Each node points to the next; the last node points to NULL. Traversal is one-directional (forward only).
- Doubly Linked List: Each node has pointers to both the next and previous nodes. Allows traversal in both directions.
- Circular Linked List: The last node connects back to the first node, forming a circle. No NULL pointer at the end.

Doubly Linked List

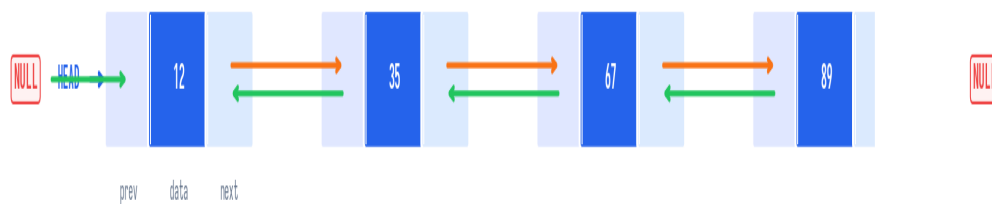


Figure 11.2: Doubly Linked List Structure

C++ Code: Singly Linked List

```

#include <iostream>
using namespace std;

struct Node {
int data;
Node* next;
Node(int val) : data(val), next(NULL) {}
};

class LinkedList {
Node* head;
public:
LinkedList() : head(NULL) {}

// Insert at the end
void insertEnd(int val) {
Node* newNode = new Node(val);
if (!head) { head = newNode; return; }
Node* temp = head;
while (temp->next) temp = temp->next;
temp->next = newNode;
}

// Delete by value
void deleteNode(int val) {
if (!head) return;
if (head->data == val) {
Node* temp = head;
head = head->next;
delete temp;
return;
}
Node* temp = head;
while (temp->next && temp->next->data != val)
temp = temp->next;
if (temp->next) {
Node* del = temp->next;
temp->next = del->next;
delete del;
}
}

// Display the list
void display() {
Node* temp = head;
while (temp) {
cout << temp->data << " -> ";
temp = temp->next;
}
cout << "NULL" << endl;
}
};

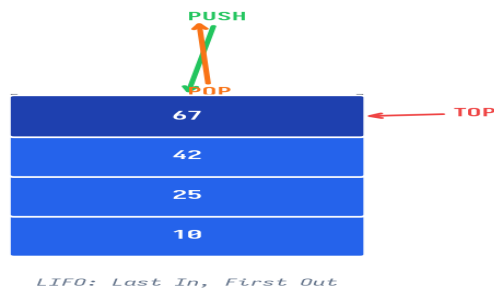
```

3. Stack

Stack: A Stack is a linear data structure that follows the Last-In-First-Out (LIFO) principle. The last element added is the first one to be removed.

Think of a stack of plates - you can only add or remove from the top.

Stack Operations



LIFO: Last In, First Out

Figure 11.3: Stack Operations

- `push(x)`: Add element `x` to the top of the stack.
- `pop()`: Remove and return the top element.
- `top()` / `peek()`: View the top element without removing it.
- `isEmpty()`: Check if the stack is empty.

Stack Overflow: When `push()` is called on a full stack (fixed-size implementation).

Stack Underflow: When `pop()` or `top()` is called on an empty stack.

C++ Code: Stack using Array

```
#include <iostream>
#define MAX 100
using namespace std;

class Stack {
int arr[MAX];
int top;
public:
Stack() : top(-1) {}

void push(int x) {
if (top >= MAX - 1) { cout << "Stack Overflow"; return; }
arr[++top] = x;
}

int pop() {
if (top < 0) { cout << "Stack Underflow"; return -1; }
return arr[top--];
}

int peek() {
if (top < 0) return -1;
return arr[top];
}
```

```
bool isEmpty() { return top < 0; }  
};
```

Real-Life Example: Undo/Redo in text editors, Browser back button, Function call stack in programming, Plate stack in cafeteria.

4. Queue

Queue: A Queue is a linear data structure that follows the First-In-First-Out (FIFO) principle. The first element added is the first one to be removed. Think of a line of people at a ticket counter - first come, first served.

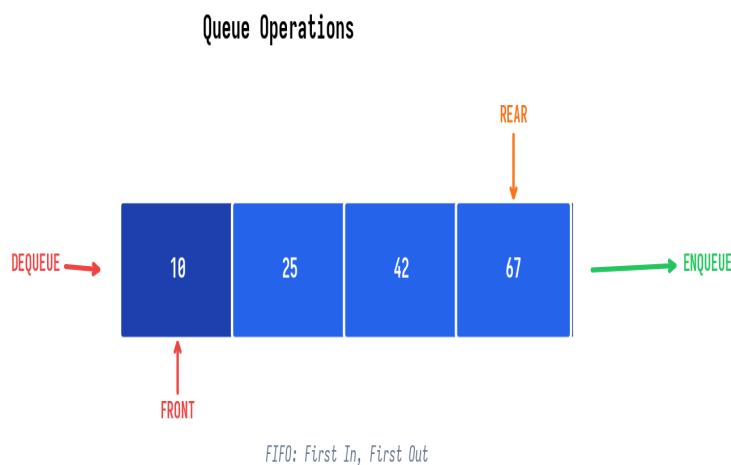


Figure 11.4: Queue Operations

- enqueue(x): Add element x to the rear of the queue.
- dequeue(): Remove and return the front element.
- front(): View the front element without removing it.
- isEmpty(): Check if the queue is empty.

Types of Queues

- Simple Queue: Standard FIFO queue.
- Circular Queue: The rear connects back to the front, making efficient use of space.
- Priority Queue: Elements are dequeued based on priority, not order of arrival.
- Double-Ended Queue (Deque): Insertion and deletion can happen at both ends.

C++ Code: Queue using Array

```
#include <iostream>  
#define MAX 100
```

```

using namespace std;

class Queue {
int arr[MAX];
int front, rear;
public:
Queue() : front(-1), rear(-1) {}

void enqueue(int x) {
if (rear >= MAX - 1) { cout << "Queue Overflow"; return; }
if (front == -1) front = 0;
arr[++rear] = x;
}

int dequeue() {
if (front == -1 || front > rear) {
cout << "Queue Underflow"; return -1;
}
return arr[front++];
}

bool isEmpty() { return front == -1 || front > rear; }
};

```

Operations Comparison

Operation	Array	Linked List	Stack	Queue
Access	O(1)	O(n)	O(n)	O(n)
Search	O(n)/O(log n)	O(n)	O(n)	O(n)
Insertion	O(n)	O(1)*	O(1) push	O(1) enqueue
Deletion	O(n)	O(1)*	O(1) pop	O(1) dequeue

* O(1) if position is known, otherwise O(n) to find the position

Important Points - Chapter 11

Key Takeaways:

1. Linear DS: Array, Linked List, Stack, Queue.
2. Linked List: Dynamic size, non-contiguous memory, O(1) insert/delete at known position.
3. Stack: LIFO - push/pop from top. Used in: undo, function calls, expression evaluation.
4. Queue: FIFO - enqueue at rear, dequeue from front. Used in: CPU scheduling, BFS.
5. Types of Queue: Simple, Circular, Priority, Deque.

Practice Questions - Chapter 11

Q1. Which data structure follows LIFO?

- (A) Queue
- (B) Stack
- (C) Array
- (D) Linked List

Answer: B

Q2. In a singly linked list, the last node points to:

- (A) First node
- (B) Itself
- (C) NULL
- (D) Previous node

Answer: C

Q3. Which queue allows insertion and deletion at both ends?

- (A) Circular Queue
- (B) Priority Queue
- (C) Dequeue
- (D) Simple Queue

Answer: C

Q4. What happens when push() is called on a full stack?

- (A) Stack Underflow
- (B) Stack Overflow
- (C) Nothing
- (D) Error message

Answer: B

Q5. Access time in an array is:

- (A) $O(n)$
- (B) $O(\log n)$
- (C) $O(1)$
- (D) $O(n \log n)$

Answer: C

Chapter 12: Trees & Graphs

Tree Data Structure

Tree: A Tree is a hierarchical, non-linear data structure used to represent data in a parent-child relationship. It consists of a root node, structural nodes, and leaf nodes connected via edges. Unlike linear structures, trees allow multiple successors per node.

Tree Terminology

Term	Definition
Root Node	Topmost node with no parent
Parent Node	Immediate predecessor of another node
Child Node	Immediate successor of another node
Leaf Node	Node with no children (external node)
Sibling	Nodes sharing the same parent
Ancestor	All nodes on path from root to the node
Descendant	All nodes reachable from a node
Level	Number of edges from root to the node (root = Level 0)
Height	Number of edges on the longest path from root to a leaf
Depth	Number of edges from root to the node
Subtree	A node and all its descendants
Internal Node	A node having at least one child
Edge	The link between a parent and child node

Table 12.1: Tree Terminology

Types of Trees

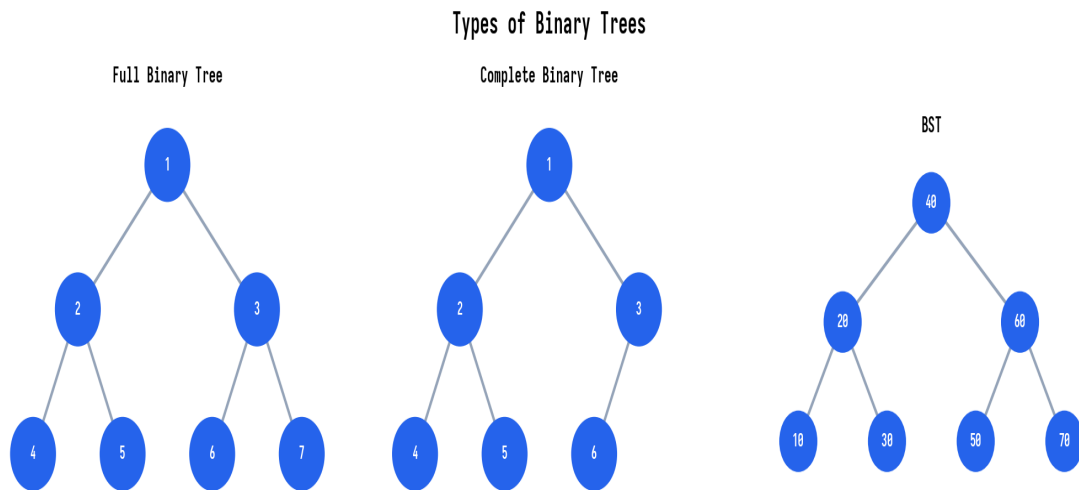


Figure 12.1: Types of Binary Trees

- Binary Tree: Each node has at most two children (left and right).
- Full Binary Tree: Every node has either 0 or 2 children. No node has only one child.
- Complete Binary Tree: All levels are completely filled except possibly the last level, which is filled from left to right.
- Perfect Binary Tree: All internal nodes have exactly 2 children and all leaves are at the same level.
- Balanced Binary Tree: The height difference between left and right subtrees of any node is at most 1.
- Binary Search Tree (BST): Left child < Parent < Right child for every node.
- AVL Tree: Self-balancing BST where balance factor of every node is -1, 0, or +1.
- Red-Black Tree: Self-balancing BST with color properties for balancing.
- B-Tree / B+ Tree: Self-balancing tree used in databases and file systems.
- Heap: Complete binary tree satisfying the heap property (max or min).

Key Tree Operations

- Create: Initialize a tree structure.
- Insert: Add data to the tree.
- Search: Check for specific data within the tree.
- Traversal: Visit every node exactly once in a specific order.
- Delete: Remove a node from the tree.

Applications of Trees

- File systems (directory structure).
- Database indexing (B-Trees, B+ Trees).
- Compiler design (syntax trees, AST).
- Networking (spanning trees, DNS hierarchy).
- Machine learning (decision trees).
- Priority queues (heaps).

Graph Data Structure

Graph: A Graph is a non-linear data structure consisting of a set of vertices (nodes) and a set of edges that connect pairs of vertices. Unlike trees, graphs can have cycles, bidirectional connections, and disconnected components.

Types of Graphs

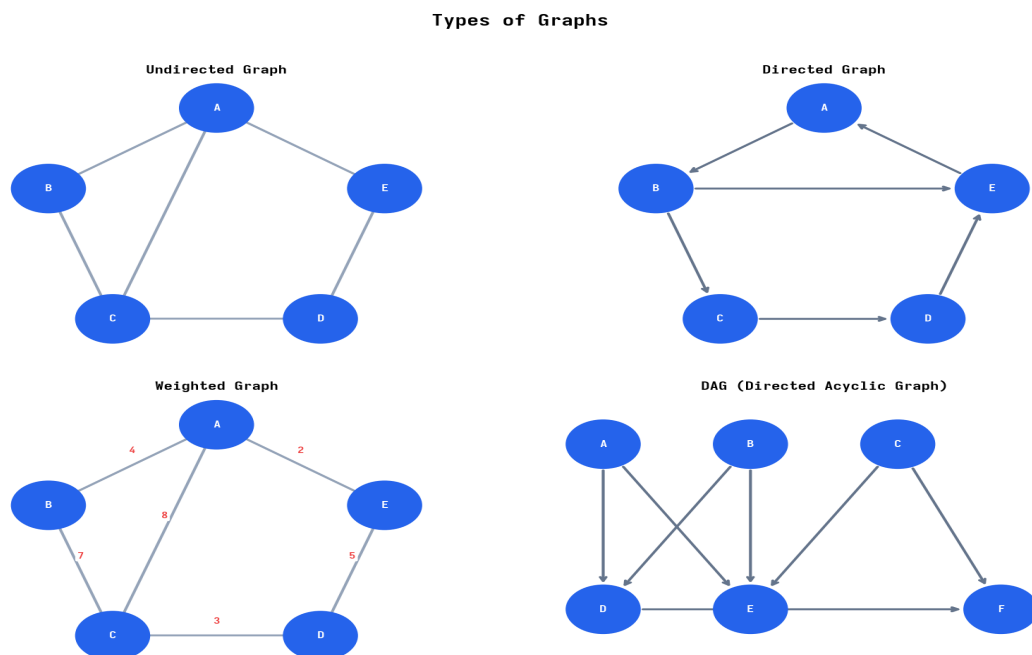


Figure 12.2: Types of Graphs

- Undirected Graph: Edges have no direction. If A-B exists, you can travel both ways. Example: Facebook friendships.
- Directed Graph (Digraph): Edges have a specific direction (shown by arrows). Example: Twitter follows (A follows B does not mean B follows A).
- Weighted Graph: Edges have values (weights) representing distance, cost, or time. Example: Google Maps (road distances).

- Unweighted Graph: Edges have no values, just connections.
- DAG (Directed Acyclic Graph): A directed graph with no cycles. Used in scheduling, dependency resolution.
- Connected Graph: Every vertex is reachable from every other vertex.
- Disconnected Graph: Some vertices are not reachable from others.
- Complete Graph: Every vertex is connected to every other vertex.
- Cyclic Graph: Contains at least one cycle (path that returns to starting vertex).
- Bipartite Graph: Vertices can be divided into two groups with edges only between groups.

Graph Properties

- Weighted: Edges have values (distance, cost, time).
- Connected: All vertices are reachable through some path.
- Directed: Edges have direction representing flow or hierarchy.
- Loop (Self-loop): An edge from a vertex to itself.
- Cyclic: Contains a cycle (path returning to the starting vertex without reusing edges).

Graph Representation

- Adjacency Matrix: 2D array where $\text{matrix}[i][j] = 1$ if edge exists. Space: $O(V^2)$. Good for dense graphs.
- Adjacency List: Array of linked lists. Each vertex stores a list of its neighbors. Space: $O(V+E)$. Good for sparse graphs.

Applications of Graphs

- Social Networks (people as nodes, friendships as edges).
- Google Maps / GPS (cities as vertices, roads as edges).
- World Wide Web (web pages as nodes, hyperlinks as edges).
- Operating Systems (resource allocation graphs).
- Circuit design, network routing, recommendation systems.

Important Points - Chapter 12

Key Takeaways:

1. Tree = Hierarchical, parent-child, one root, no cycles.
2. Key terms: root, leaf, parent, child, sibling, ancestor, height, depth, level.

3. Binary Tree types: Full, Complete, Perfect, Balanced, BST, AVL, Heap.
4. Graph = Vertices + Edges. Can have cycles, directions, weights.
5. Graph types: Directed, Undirected, Weighted, DAG, Complete, Bipartite.
6. Representations: Adjacency Matrix (dense) vs Adjacency List (sparse).

Practice Questions - Chapter 12

Q1. The topmost node of a tree is called:

- (A) Leaf
- (B) Root
- (C) Parent
- (D) Internal node

Answer: B

Q2. A graph with no cycles is called:

- (A) Complete graph
- (B) Cyclic graph
- (C) Acyclic graph
- (D) Bipartite graph

Answer: C

Q3. In a full binary tree, each node has:

- (A) 0 or 1 children
- (B) 0 or 2 children
- (C) Exactly 2 children
- (D) 1 or 2 children

Answer: B

Q4. Adjacency Matrix uses how much space?

- (A) $O(V)$
- (B) $O(E)$
- (C) $O(V+E)$
- (D) $O(V^2)$

Answer: D

Q5. A DAG is a graph that is:

- (A) Directed and Cyclic
- (B) Directed and Acyclic
- (C) Undirected and Cyclic
- (D) Undirected and Acyclic

Answer: B

Chapter 13: Binary Tree & Binary Search Tree

Binary Tree

Binary Tree: A Binary Tree is a non-linear, hierarchical data structure where each node has at most two children, referred to as the left child and the right child. The topmost node is called the root, and the bottom-most nodes with no children are called leaves.

Binary Search Tree

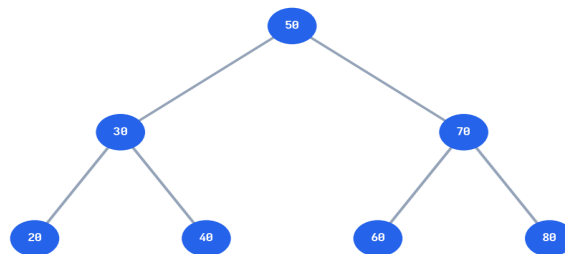


Figure 13.1: Binary Search Tree Example

Binary Tree Terminology

Term	Definition
Root	Topmost node in the tree
Leaf	Node with no children
Edge	Link between parent and child
Path	Sequence of nodes connected by edges
Depth of Node	Number of edges from root to that node
Height of Tree	Number of edges on the longest path from root to leaf
Subtree	A node and all its descendants
Internal Node	A node with at least one child

C++ Code: Binary Tree Node

```
#include <iostream>
using namespace std;

struct Node {
int data;
Node* left;
Node* right;
Node(int val) : data(val), left(NULL), right(NULL) {}
};

int main() {
// Create a simple binary tree
// 1
// / \
// 2 3
// /
// 4
Node* root = new Node(1);
root->left = new Node(2);
root->right = new Node(3);
root->left->left = new Node(4);
return 0;
}
```

Binary Search Tree (BST)

Binary Search Tree (BST): A Binary Search Tree is a special type of binary tree that maintains elements in sorted order. For every node: (1) All nodes in its left subtree have values LESS than the node, and (2) All nodes in its right subtree have values GREATER than the node.

The BST property enables efficient searching: at each node, you can eliminate half of the remaining tree, achieving $O(\log n)$ search time in a balanced tree. However, if the tree becomes skewed (all nodes on one side), search degrades to $O(n)$, which is why balanced BST variants (AVL, Red-Black) are important.

BST Operations

Search in BST

1. Start at the root.
2. If $key == root$, found!
3. If $key < root$, search left subtree.
4. If $key > root$, search right subtree.
5. If NULL is reached, key not found.

Insert in BST

1. Start at the root.
2. If $\text{key} < \text{root}$, go left. If left is NULL, insert there.
3. If $\text{key} > \text{root}$, go right. If right is NULL, insert there.
4. Duplicate keys are typically not allowed or stored in a specific subtree.

C++ Code: BST Operations

```
#include <iostream>
using namespace std;

struct Node {
    int data;
    Node* left, *right;
    Node(int val) : data(val), left(NULL), right(NULL) {}
};

class BST {
    Node* root;
public:
    BST() : root(NULL) {}

    // Insert a value into BST
    Node* insert(Node* node, int val) {
        if (!node) return new Node(val);
        if (val < node->data)
            node->left = insert(node->left, val);
        else if (val > node->data)
            node->right = insert(node->right, val);
        return node;
    }

    // Search for a value in BST
    Node* search(Node* node, int key) {
        if (!node || node->data == key) return node;
        if (key < node->data)
            return search(node->left, key);
        return search(node->right, key);
    }

    // Inorder traversal (gives sorted output)
    void inorder(Node* node) {
        if (!node) return;
        inorder(node->left);
        cout << node->data << " ";
        inorder(node->right);
    }
};
```

Time Complexity - BST Operations

Operation	Average Case	Worst Case (Skewed)
Search	$O(\log n)$	$O(n)$

Insert	$O(\log n)$	$O(n)$
Delete	$O(\log n)$	$O(n)$
Traversal	$O(n)$	$O(n)$

Note: BST operations are $O(\log n)$ ONLY when the tree is balanced. In the worst case (skewed tree), they degrade to $O(n)$. This is why self-balancing trees like AVL and Red-Black trees are used in practice.

Advantages & Disadvantages of BST

Advantages:

- Efficient search $O(\log n)$ when balanced.
- Inorder traversal gives sorted output.
- Dynamic size - grows as needed.
- Easy to implement.

Disadvantages:

- Can become skewed ($O(n)$ operations).
- No guarantee of balance without additional mechanisms.

Important Points - Chapter 13

Key Takeaways:

1. Binary Tree: Each node has at most 2 children.
2. BST: $\text{Left} < \text{Parent} < \text{Right}$ for every node.
3. BST Search/Insert/Delete: $O(\log n)$ avg, $O(n)$ worst (skewed).
4. Inorder traversal of BST gives sorted output.
5. BST performance depends on balance. Skewed = $O(n)$.
6. Solution: Use AVL or Red-Black trees for guaranteed balance.

Practice Questions - Chapter 13

Q1. In a BST, all nodes in the left subtree are:

- (A) Greater than root
- (B) Less than root
- (C) Equal to root
- (D) Random

Answer: B

Q2. Inorder traversal of a BST gives:

- (A) Reverse sorted
- (B) Random order
- (C) Sorted ascending
- (D) Sorted descending

Answer: C

Q3. Worst case BST search complexity:

- (A) $O(1)$
- (B) $O(\log n)$
- (C) $O(n)$
- (D) $O(n \log n)$

Answer: C

Q4. When does BST degenerate to $O(n)$?

- (A) When balanced
- (B) When skewed
- (C) When full
- (D) When complete

Answer: B

Q5. A BST node has at most how many children?

- (A) 1
- (B) 2
- (C) 3
- (D) Unlimited

Answer: B

Chapter 14: Tree Traversal Techniques

Tree Traversal: Tree Traversal is the process of visiting or accessing each node of a tree exactly once in a specific order. Unlike linear data structures which have one logical traversal, trees offer multiple methods categorized into Depth-First Search (DFS) and Breadth-First Search (BFS).

Types of Traversals

1. Depth First Search (DFS):

- Inorder Traversal (Left -> Root -> Right)
- Preorder Traversal (Root -> Left -> Right)

- Postorder Traversal (Left -> Right -> Root)
2. Breadth First Search (BFS):
- Level Order Traversal (Level by Level, left to right)

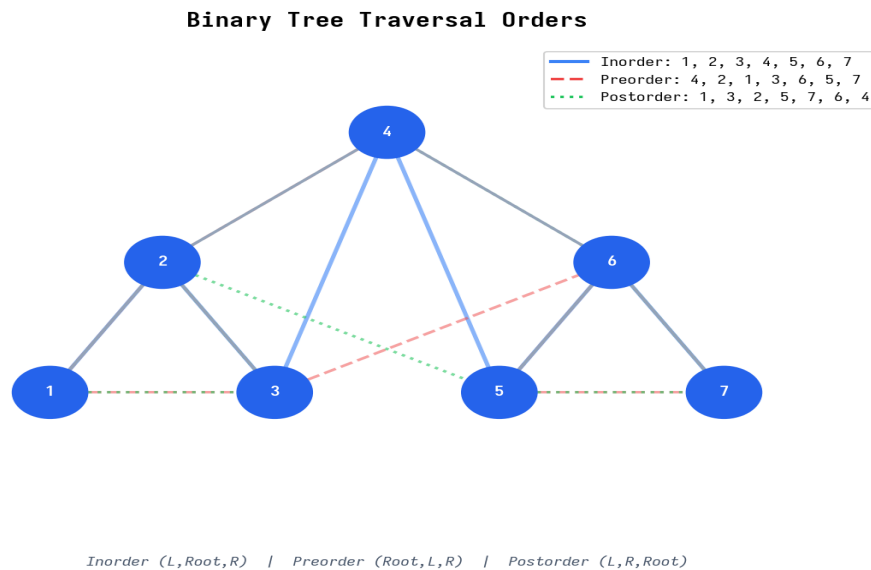


Figure 14.1: Tree Traversal Orders

Example Tree

Consider this binary tree:

```

1
/ \
2 3
/ \ \
4 5 6

```

1. Inorder Traversal (Left -> Root -> Right)

Steps: (1) Traverse left subtree recursively, (2) Visit the root, (3) Traverse right subtree recursively.

Dry Run on the example tree:

1. Go to node 1, traverse left subtree first.
2. At node 2, traverse left: visit 4. Visit 2. Traverse right: visit 5.
3. Back at node 1, visit 1.
4. Traverse right subtree: go to 3, left is NULL, visit 3, traverse right: visit 6.

Result: 4, 2, 5, 1, 3, 6

Note: For a BST, Inorder traversal ALWAYS produces elements in sorted (ascending) order. This is a key property!

2. Preorder Traversal (Root -> Left -> Right)

Steps: (1) Visit the root, (2) Traverse left subtree recursively, (3) Traverse right subtree recursively.

Dry Run: Visit 1 -> Visit 2 -> Visit 4 -> Visit 5 -> Visit 3 -> Visit 6

Result: 1, 2, 4, 5, 3, 6

Tip / Trick: Trick: Preorder = "Pre" means Root comes first. Pre = Root first.

3. Postorder Traversal (Left -> Right -> Root)

Steps: (1) Traverse left subtree recursively, (2) Traverse right subtree recursively, (3) Visit the root.

Dry Run: Visit 4 -> Visit 5 -> Visit 2 -> Visit 6 -> Visit 3 -> Visit 1

Result: 4, 5, 2, 6, 3, 1

Tip / Trick: Trick: Postorder = "Post" means Root comes last (post = after). Post = Root last.

4. Level Order Traversal (BFS)

Level Order Traversal visits all nodes at the current level completely before moving to the next level. It uses a Queue data structure.

Algorithm:

1. Create an empty queue.
2. Enqueue the root node.
3. While queue is not empty:
 - a. Dequeue a node and visit it.
 - b. Enqueue its left child if it exists.
 - c. Enqueue its right child if it exists.

Dry Run:

Queue: [1] -> Visit 1, Enqueue 2,3. Queue: [2,3]

Queue: [2,3] -> Visit 2, Enqueue 4,5. Queue: [3,4,5]

Queue: [3,4,5] -> Visit 3, Enqueue 6. Queue: [4,5,6]

Queue: [4,5,6] -> Visit 4. Queue: [5,6] -> Visit 5. Queue: [6] -> Visit 6.

Result: 1, 2, 3, 4, 5, 6

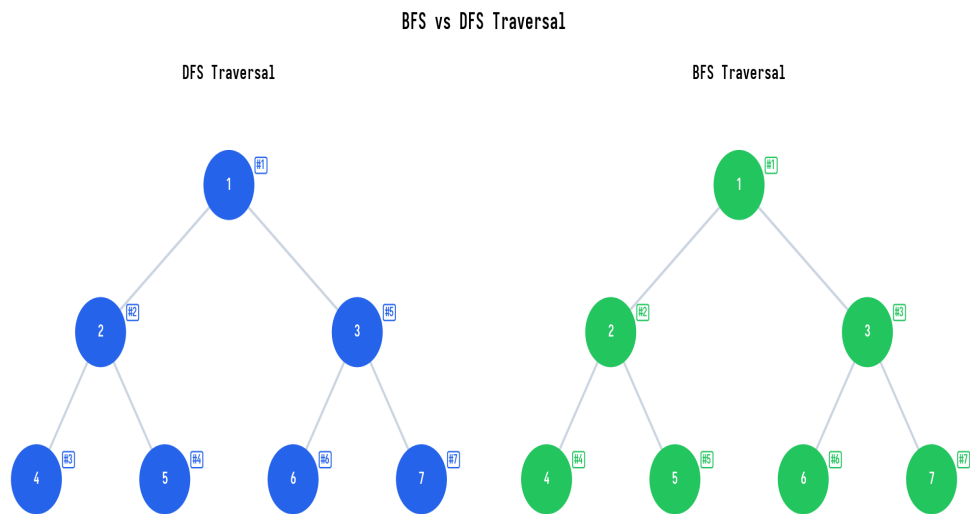


Figure 14.2: BFS and DFS Traversal Comparison

C++ Code: All Traversals

```
#include <iostream>
#include <queue>
using namespace std;

struct Node {
    int data;
    Node* left, *right;
    Node(int val) : data(val), left(NULL), right(NULL) {}
};

// Inorder: Left -> Root -> Right
void inorder(Node* root) {
    if (!root) return;
    inorder(root->left);
    cout << root->data << " ";
    inorder(root->right);
}

// Preorder: Root -> Left -> Right
void preorder(Node* root) {
    if (!root) return;
    cout << root->data << " ";
    preorder(root->left);
    preorder(root->right);
}

// Postorder: Left -> Right -> Root
void postorder(Node* root) {
    if (!root) return;
    postorder(root->left);
    postorder(root->right);
    cout << root->data << " ";
```

```

}

// Level Order: BFS using Queue
void levelOrder(Node* root) {
    if (!root) return;
    queue<Node*> q;
    q.push(root);
    while (!q.empty()) {
        Node* curr = q.front();
        q.pop();
        cout << curr->data << " ";
        if (curr->left) q.push(curr->left);
        if (curr->right) q.push(curr->right);
    }
}

```

Traversal Summary

Traversal	Order	Use Case
Inorder	Left-Root-Right	BST: gives sorted output
Preorder	Root-Left-Right	Copy tree, prefix expression
Postorder	Left-Right-Root	Delete tree, postfix expression
Level Order	Level by Level	BFS, shortest path in unweighted tree

Tip / Trick: Easy Memory Trick: In-Order = Root IN the middle. Pre-Order = Root BEFORE (pre). Post-Order = Root AFTER (post).

Important Points - Chapter 14

Key Takeaways:

1. DFS: Inorder (L-Rt-R), Preorder (Rt-L-R), Postorder (L-R-Rt).
2. BFS: Level Order - uses Queue, visits level by level.
3. Inorder of BST = Sorted output (key property!).
4. Preorder = Copy tree, Prefix expression. Postorder = Delete tree, Postfix.
5. All traversals: $O(n)$ time, $O(h)$ space for DFS (recursion stack), $O(w)$ for BFS (queue width).
6. Trick: IN=Root IN middle, PRE=Root BEFORE, POST=Root AFTER.

Practice Questions - Chapter 14

Q1. Which traversal of BST gives sorted output?

- (A) Preorder
- (B) Inorder

- (C) Postorder
- (D) Level Order

Answer: B

Q2. Level Order Traversal uses which data structure?

- (A) Stack
- (B) Queue
- (C) Array
- (D) Linked List

Answer: B

Q3. In Preorder traversal, the root is visited:

- (A) First
- (B) Second
- (C) Last
- (D) Middle

Answer: A

Q4. Postorder traversal is useful for:

- (A) Copying a tree
- (B) Deleting a tree
- (C) Searching a tree
- (D) Balancing a tree

Answer: B

Q5. The time complexity of all traversals is:

- (A) $O(1)$
- (B) $O(\log n)$
- (C) $O(n)$
- (D) $O(n^2)$

Answer: C

Chapter 15: Balanced Binary Tree

Balanced Binary Tree: A binary tree is balanced if for every node, the absolute difference between the heights of the left and right subtrees is at most 1. This ensures that the height of the tree remains $O(\log n)$, guaranteeing efficient $O(\log n)$ operations.

Balanced binary trees are crucial for maintaining the $O(\log n)$ search, insert, and delete performance that makes binary search trees useful. Without balancing, a

BST can degenerate into a linked list (skewed tree) with $O(n)$ operations. Self-balancing trees like AVL trees and Red-Black trees automatically maintain balance after every insertion and deletion, ensuring that operations never degrade beyond $O(\log n)$.

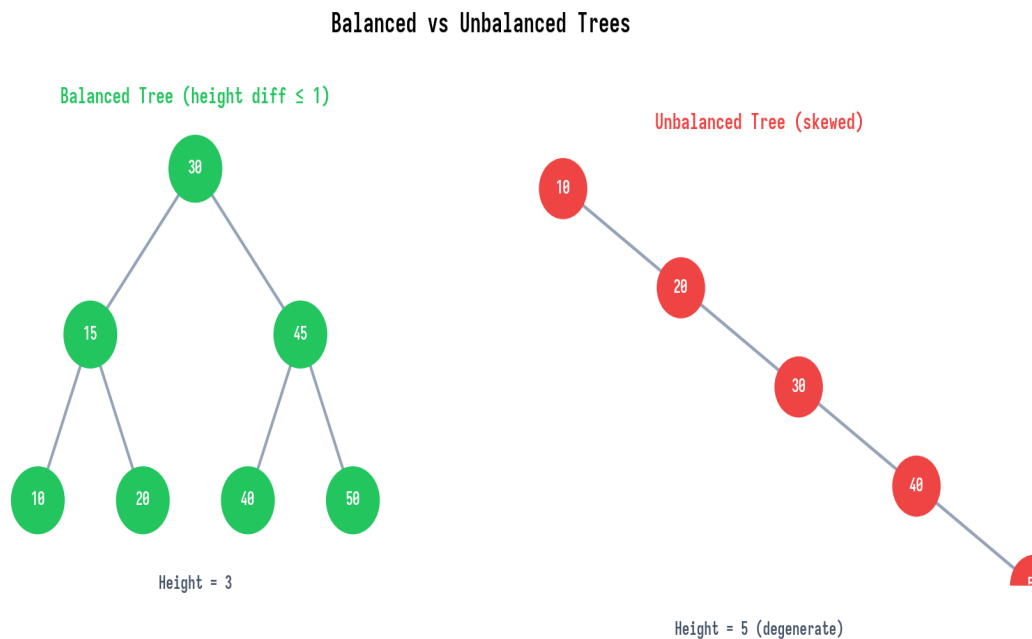


Figure 15.1: Balanced vs Unbalanced Tree

Balance Factor

Balance Factor: The Balance Factor of a node is defined as: Height of Left Subtree - Height of Right Subtree. For a balanced tree, the balance factor of every node must be -1, 0, or +1.

Balance Factor	Meaning	Status
0	Left and right subtrees have equal height	Balanced
+1	Left subtree is one level taller	Balanced
-1	Right subtree is one level taller	Balanced
+2 or more	Left subtree is too tall	UNBALANCED
-2 or less	Right subtree is too tall	UNBALANCED

How to Check if a Tree is Balanced

A binary tree is balanced if and only if ALL three conditions are met:

- 1. The absolute difference between heights of left and right subtrees at the root is at most 1.
- 2. The left subtree is itself a balanced binary tree.

- 3. The right subtree is itself a balanced binary tree.

Note: This is a recursive definition - every subtree must also be balanced!

C++ Code: Check if Tree is Balanced

```
#include <iostream>
#include <algorithm>
#include <cmath>
using namespace std;

struct Node {
int data;
Node* left, *right;
Node(int val) : data(val), left(NULL), right(NULL) {}
};

// Calculate height of tree
int height(Node* node) {
if (!node) return 0;
return 1 + max(height(node->left), height(node->right));
}

// Check if tree is balanced
bool isBalanced(Node* root) {
if (!root) return true;
int leftH = height(root->left);
int rightH = height(root->right);
// Check balance at current node and recursively for subtrees
return (abs(leftH - rightH) <= 1) &&
isBalanced(root->left) &&
isBalanced(root->right);
}
```

Advantages of Balanced Trees

- Logarithmic Efficiency: Balanced trees maintain $O(\log n)$ height, ensuring $O(\log n)$ for search, insert, and delete operations.
- Non-Destructive Operations: Operations maintain balance without requiring a complete reorganization of the tree.
- Range Queries: Well-suited for finding all elements within a specified range efficiently.
- Predictable Performance: Unlike unbalanced BSTs, performance does not degrade based on insertion order.

Important Points - Chapter 15

Key Takeaways:

1. Balanced Tree: $|\text{height}(\text{left}) - \text{height}(\text{right})| \leq 1$ for EVERY node.

2. Balance Factor = Left Height - Right Height. Valid: -1, 0, +1.
3. Balanced tree height = $O(\log n)$. Unbalanced can be $O(n)$.
4. Self-balancing trees: AVL (strict) and Red-Black (relaxed).
5. Balance check is recursive: every subtree must be balanced.

Practice Questions - Chapter 15

Q1. A balanced binary tree has balance factor:

- (A) 0 only
- (B) -1, 0, or +1
- (C) Any positive number
- (D) Any negative number

Answer: B

Q2. An unbalanced BST can have $O(n)$ operations because it becomes:

- (A) A heap
- (B) A linked list (skewed)
- (C) A graph
- (D) A complete tree

Answer: B

Q3. The height of a balanced tree with n nodes is:

- (A) $O(n)$
- (B) $O(n^2)$
- (C) $O(\log n)$
- (D) $O(1)$

Answer: C

Q4. Which are self-balancing trees?

- (A) BST
- (B) AVL and Red-Black
- (C) Heap
- (D) B-Tree only

Answer: B

Q5. Balance factor is calculated as:

- (A) Right height - Left height
- (B) Left height - Right height
- (C) Left height + Right height
- (D) Left height * Right height

Answer: B

Chapter 16: AVL Tree

AVL Tree: An AVL Tree (Adelson-Velsky and Landis Tree) is a self-balancing Binary Search Tree where the difference between heights of left and right subtrees for any node cannot be more than one. After every insertion or deletion, the tree rebalances itself using rotations to maintain this property.

AVL trees were the first self-balancing BST invented (1962). They maintain strict balance, which means the height of an AVL tree with n nodes is always $O(\log n)$. This guarantees $O(\log n)$ time for search, insert, and delete operations. The stricter balancing of AVL trees makes them faster for lookup-heavy applications compared to Red-Black trees, but insertion and deletion require more rotations.

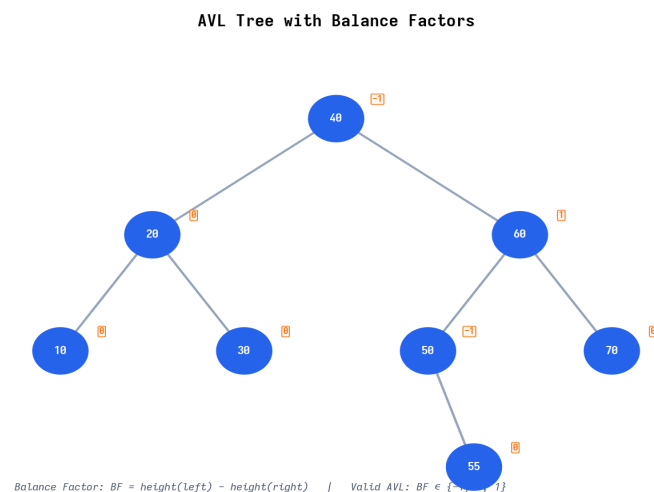


Figure 16.1: AVL Tree with Balance Factors

AVL Tree Properties

- It is a BST (left < parent < right).
- Balance Factor of every node is -1, 0, or +1.
- Balance Factor = height(left subtree) - height(right subtree).
- After insertion or deletion, the tree automatically rebalances using rotations.
- Height is always $O(\log n)$.
- Search is identical to normal BST but guaranteed $O(\log n)$.

AVL Rotations (CRITICAL - Exam Topic!)

When an insertion or deletion causes the balance factor of any node to become +2 or -2, the tree becomes unbalanced. Rotations are performed to restore balance.

There are four types of imbalances and their corresponding rotations:

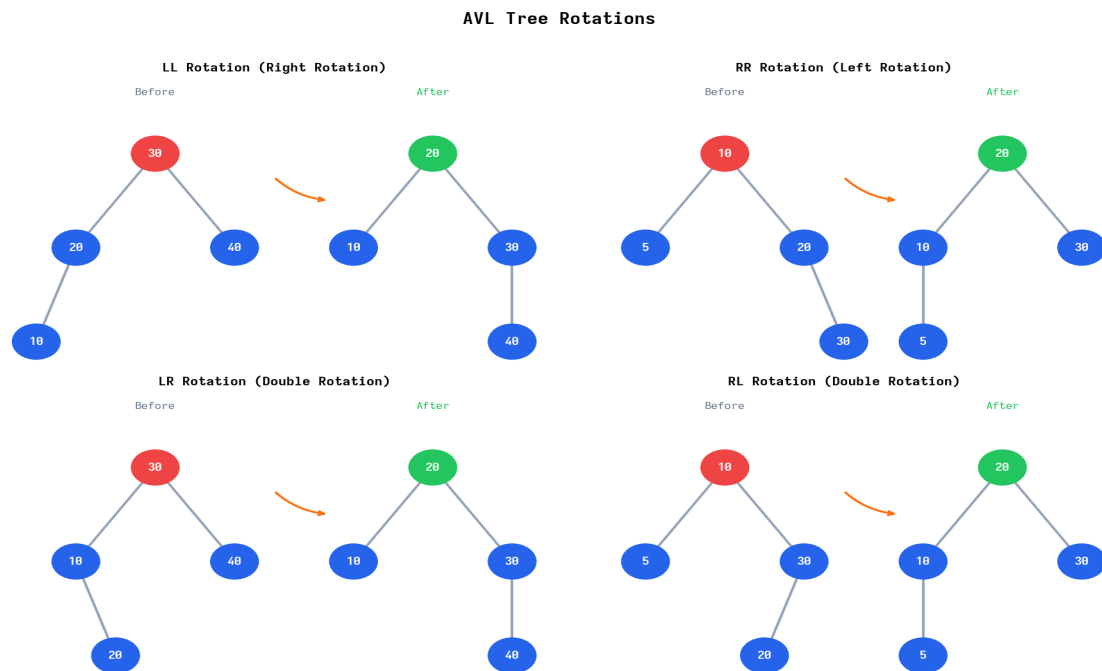


Figure 16.2: AVL Tree Rotations (LL, RR, LR, RL)

1. LL Rotation (Left-Left Case)

When it happens: A node is inserted into the LEFT subtree of the LEFT child of an unbalanced node. The balance factor becomes +2.

Fix: Perform a SINGLE RIGHT ROTATION on the unbalanced node.

Dry Run - LL Rotation:

```
Before Insertion: After Inserting 5:
20 (BF=+1) 20 (BF=+2) <-- Unbalanced!
/ /
10 (BF=0) 10 (BF=+1)
/
5 (BF=0)

After RIGHT ROTATION on 20:
10 (BF=0)
/ \
5 20
All balance factors are now valid!
```

2. RR Rotation (Right-Right Case)

When it happens: A node is inserted into the RIGHT subtree of the RIGHT child of an unbalanced node. The balance factor becomes -2.

Fix: Perform a SINGLE LEFT ROTATION on the unbalanced node.

Dry Run - RR Rotation:

```
Before: After Inserting 40:
20 (BF=-1) 20 (BF=-2) <-- Unbalanced!
\  \
30 (BF=0) 30 (BF=-1)
\
40

After LEFT ROTATION on 20:
30 (BF=0)
/  \
20 40
Balanced!
```

3. LR Rotation (Left-Right Case)

When it happens: A node is inserted into the RIGHT subtree of the LEFT child of an unbalanced node. The balance factor becomes +2.

Fix: DOUBLE ROTATION - First LEFT rotate the left child, then RIGHT rotate the unbalanced node.

Dry Run - LR Rotation:

```
After Inserting 15 into:
20 (BF=+2) <-- Unbalanced!
/
10 (BF=-1)
\
15

Step 1: LEFT Rotate on 10:
20
/
15
/
10

Step 2: RIGHT Rotate on 20:
15 (BF=0)
/  \
10 20
Balanced!
```

4. RL Rotation (Right-Left Case)

When it happens: A node is inserted into the LEFT subtree of the RIGHT child of an unbalanced node. The balance factor becomes -2.

Fix: DOUBLE ROTATION - First RIGHT rotate the right child, then LEFT rotate the unbalanced node.

Dry Run - RL Rotation:

```

After Inserting 25 into:
20 (BF=-2) <-- Unbalanced!
\
30 (BF=+1)
/
25

Step 1: RIGHT Rotate on 30:
20
\
25
\
30

Step 2: LEFT Rotate on 20:
25 (BF=0)
/ \
20 30
Balanced!

```

Rotation Summary Table

Case	Insertion Location	Balance Factor	Rotation Needed
LL	Left of Left child	+2	Single Right Rotation
RR	Right of Right child	-2	Single Left Rotation
LR	Right of Left child	+2	Left on left child, then Right on node
RL	Left of Right child	-2	Right on right child, then Left on node

Tip / Trick: Trick to Remember: LL = Right rotation (opposite of L). RR = Left rotation (opposite of R). LR = Left then Right. RL = Right then Left. The rotation direction is always the OPPOSITE of the imbalance direction!

C++ Code: AVL Tree

```

#include <iostream>
#include <algorithm>
using namespace std;

struct Node {
    int data, height;
    Node* left, *right;
    Node(int val) : data(val), height(1),
    left(NULL), right(NULL) {}
};

int getHeight(Node* node) {
    return node ? node->height : 0;
}

int getBalance(Node* node) {

```

```

return node ? getHeight(node->left) - getHeight(node->right) : 0;
}

// Right Rotation (for LL case)
Node* rightRotate(Node* y) {
Node* x = y->left;
Node* T2 = x->right;
x->right = y; // Perform rotation
y->left = T2;
// Update heights
y->height = 1 + max(getHeight(y->left), getHeight(y->right));
x->height = 1 + max(getHeight(x->left), getHeight(x->right));
return x; // New root
}

// Left Rotation (for RR case)
Node* leftRotate(Node* x) {
Node* y = x->right;
Node* T2 = y->left;
y->left = x; // Perform rotation
x->right = T2;
// Update heights
x->height = 1 + max(getHeight(x->left), getHeight(x->right));
y->height = 1 + max(getHeight(y->left), getHeight(y->right));
return y; // New root
}

// Insert into AVL Tree
Node* insert(Node* node, int val) {
// 1. Standard BST insertion
if (!node) return new Node(val);
if (val < node->data)
node->left = insert(node->left, val);
else if (val > node->data)
node->right = insert(node->right, val);
else return node; // No duplicates

// 2. Update height
node->height = 1 + max(getHeight(node->left),
getHeight(node->right));

// 3. Get balance factor
int balance = getBalance(node);

// 4. Perform rotations if unbalanced
// LL Case
if (balance > 1 && val < node->left->data)
return rightRotate(node);
// RR Case
if (balance < -1 && val > node->right->data)
return leftRotate(node);
// LR Case
if (balance > 1 && val > node->left->data) {
node->left = leftRotate(node->left);
return rightRotate(node);
}
// RL Case
if (balance < -1 && val < node->right->data) {

```

```

node->right = rightRotate(node->right);
return leftRotate(node);
}
return node;
}

```

Complete AVL Insertion Dry Run

Insert sequence: 10, 20, 30, 40, 50, 25

Insert 10: Tree: 10 (BF=0) - Balanced

Insert 20: Tree: 10(BF=-1) -> 20(BF=0) - Balanced

Insert 30: 10(BF=-2) -> unbalanced! RR case. Left rotate on 10.

After rotation: 20(BF=0) with left=10, right=30. Balanced!

```

20 (BF=0)
/ \
10 30

```

Insert 40: 30(BF=-1), 20(BF=-1) - Balanced

Insert 50: 30(BF=-2) -> unbalanced! RR case. Left rotate on 30.

After rotation: 20(BF=-1) -> left=10, right=40; 40(BF=0) -> left=30, right=50

```

20 (BF=-1)
/ \
10 40 (BF=0)
/ \
30 50

```

Insert 25: 40(BF=+1), 20(BF=-2) -> unbalanced! RL case.

Step 1: Right rotate on 40. Step 2: Left rotate on 20.

```

30 (BF=0)
/ \
20 40 (BF=-1)
/ \ \
10 25 50

```

Final AVL Tree - All balance factors valid!

Time Complexity - AVL Tree

Operation	Time Complexity	Note
Search	$O(\log n)$	Same as BST but guaranteed $O(\log n)$
Insert	$O(\log n)$	BST insert + $O(1)$ rotation(s)

Delete	$O(\log n)$	May require multiple rotations
Space	$O(n)$	Stores height in each node

Advantages & Disadvantages

Advantages:

- Self-balancing: $O(\log n)$ guaranteed for all operations.
- Faster lookups than Red-Black trees due to stricter balancing.
- Inorder traversal gives sorted order.

Disadvantages:

- More complex implementation than regular BST.
- More rotations during insertion/deletion than Red-Black trees.
- Less commonly used in standard libraries (Red-Black preferred).

Important Points - Chapter 16

Key Takeaways:

1. AVL = Self-balancing BST. Balance Factor = $\{-1, 0, +1\}$.
2. Four rotation cases: LL (Right rotate), RR (Left rotate), LR (Left then Right), RL (Right then Left).
3. Trick: Rotation = opposite of imbalance. LL->Right, RR->Left, LR->Left+Right, RL->Right+Left.
4. All operations $O(\log n)$. Rotation takes $O(1)$.
5. Stricter balance = faster search but more rotations than Red-Black.
6. AVL trees are the first self-balancing BST (1962).

Practice Questions - Chapter 16

Q1. In LL case, which rotation is performed?

- (A) Left
- (B) Right
- (C) Left then Right
- (D) Right then Left

Answer: B

Q2. What is the valid range of balance factor in AVL?

- (A) -2 to +2
- (B) -1 to +1
- (C) 0 to 1

(D) 0 only

Answer: B

Q3. LR rotation is:

(A) Left only

(B) Right only

(C) Left then Right

(D) Right then Left

Answer: C

Q4. After inserting 10,20,30 in AVL, what rotation is needed?

(A) LL

(B) RR

(C) LR

(D) RL

Answer: B

Q5. AVL tree search complexity:

(A) $O(n)$

(B) $O(\log n)$

(C) $O(n \log n)$

(D) $O(1)$

Answer: B

Q6. In RL case, first rotation is on the:

(A) Unbalanced node

(B) Left child

(C) Right child

(D) Root

Answer: C

Q7. AVL stands for:

(A) Adaptive Variable Length

(B) Adelson-Velsky Landis

(C) Automatic Vector Layout

(D) Algorithm Verification Logic

Answer: B

Q8. Compared to Red-Black trees, AVL trees have:

(A) Fewer rotations

(B) More rotations

(C) Same rotations

(D) No rotations

Answer: B

Chapter 17: Shortest Path Algorithms

Shortest Path Problem: The shortest path problem means finding the shortest possible route or path between two vertices in a Graph, where edges have weights representing distance, time, or cost.

Shortest path algorithms are essential in many real-world applications. Navigation systems like Google Maps use them to find the fastest route. Network routing protocols use them to find optimal data paths. Social networks use them to find degrees of separation. Two major algorithms solve this problem: Dijkstra's Algorithm and Bellman-Ford Algorithm.

Dijkstra's Algorithm

Dijkstra's Algorithm: Dijkstra's Algorithm finds the shortest path from one source vertex to all other vertices in a weighted graph with non-negative edge weights. It works by repeatedly selecting the nearest unvisited vertex and updating distances to its neighbors.

Dijkstra's Algorithm - Shortest Path

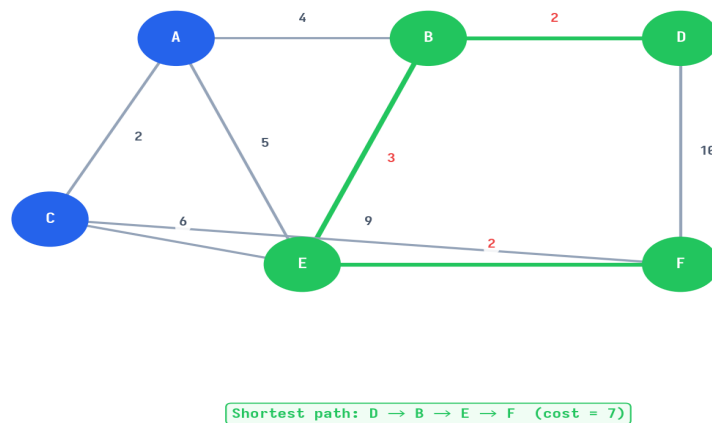


Figure 17.1: Weighted Graph for Shortest Path

Algorithm Steps

1. Initialize: Set source distance = 0, all other distances = infinity.
2. Mark all vertices as unvisited.
3. Select the unvisited vertex with the smallest distance.
4. For each unvisited neighbor, calculate the tentative distance through the

current vertex.

5. If tentative distance < current distance, update the distance.
6. Mark the current vertex as visited.
7. Repeat steps 3-6 until all vertices are visited.

Dry Run: Dijkstra's Algorithm

Graph: 6 vertices (D, A, E, C, G, B, F). Source = D.

Step	Selected	Distance Updates	Visited
1	D (dist=0)	A=4, E=2	D
2	E (dist=2)	C=6(2+4), G=7(2+5)	D,E
3	A (dist=4)	C: 4+3=7 > 6 (no update)	D,E,A
4	C (dist=6)	B=8(6+2), F=11(6+5)	D,E,A,C
5	G (dist=7)	F: 7+5=12 > 11 (no update)	D,E,A,C,G
6	B (dist=8)	F: 8+2=10 < 11 (update F=10)	D,E,A,C,G,B
7	F (dist=10)	All visited!	All

Final distances from D: A=4, B=8, C=6, E=2, F=10, G=7

Complexity - Dijkstra

Time Complexity: $O(E \log V)$ using a min-heap/priority queue, where E = number of edges and V = number of vertices. Without a heap, it is $O(V^2)$.

Space Complexity: $O(V)$ for storing distances, visited set, and the priority queue.

Note: Dijkstra's Algorithm does NOT work with negative edge weights. Use Bellman-Ford for graphs with negative weights.

Bellman-Ford Algorithm

Bellman-Ford Algorithm: Bellman-Ford is a single-source shortest path algorithm that can handle graphs with negative edge weights. It works by relaxing all edges repeatedly for $V-1$ iterations, where V is the number of vertices. It can also detect negative weight cycles.

While slower than Dijkstra's algorithm, Bellman-Ford is more versatile because it can handle negative edge weights. It is also used to detect negative weight cycles in a graph, which Dijkstra cannot do. The algorithm relaxes all edges in each of

V-1 passes, ensuring that the shortest paths are found even when negative weights are present.

Algorithm Steps

1. Initialize: Set source distance = 0, all others = infinity.
2. Repeat V-1 times: For each edge (u, v, weight), if $\text{dist}[u] + \text{weight} < \text{dist}[v]$, update $\text{dist}[v]$.
3. (Optional) Check for negative cycle: If any edge can still be relaxed, a negative weight cycle exists.

Complexity - Bellman-Ford

Time Complexity: $O(V \times E)$, where V = number of vertices and E = number of edges. This is slower than Dijkstra but handles negative weights.

Space Complexity: $O(V)$ for storing the distance array.

Dijkstra vs Bellman-Ford

Feature	Dijkstra	Bellman-Ford
Time Complexity	$O(E \log V)$	$O(V \times E)$
Negative Weights	Not Supported	Supported
Negative Cycle Detection	No	Yes
Approach	Greedy	Dynamic Programming
Data Structure	Priority Queue	Edge List
Speed	Faster	Slower

Important Points - Chapter 17

Key Takeaways:

1. Dijkstra: Greedy, $O(E \log V)$, NO negative weights. Use priority queue.
2. Bellman-Ford: DP, $O(V \times E)$, handles negative weights, detects negative cycles.
3. Both find single-source shortest paths.
4. Dijkstra is faster but limited. Bellman-Ford is slower but more versatile.
5. Real-world: GPS = Dijkstra, Financial networks with losses = Bellman-Ford.

Practice Questions - Chapter 17

Q1. Dijkstra's algorithm cannot handle:

- (A) Large graphs
- (B) Negative weights
- (C) Directed graphs
- (D) Weighted graphs

Answer: B

Q2. Time complexity of Bellman-Ford is:

- (A) $O(E \log V)$
- (B) $O(V \times E)$
- (C) $O(V^2)$
- (D) $O(E)$

Answer: B

Q3. Which algorithm can detect negative weight cycles?

- (A) Dijkstra
- (B) Bellman-Ford
- (C) Both
- (D) Neither

Answer: B

Q4. Dijkstra uses which data structure?

- (A) Queue
- (B) Stack
- (C) Priority Queue
- (D) Hash Table

Answer: C

Q5. Bellman-Ford relaxes all edges how many times?

- (A) V
- (B) $V-1$
- (C) E
- (D) $E-1$

Answer: B

Chapter 18: Prefix / Postfix String Manipulation

In this chapter, we learn how to separate operands and operators from a mathematical expression using string manipulation, and generate simplified prefix and postfix representations through character grouping. This is a foundational concept for understanding expression evaluation and conversion between notation systems.

Types of Notation

Infix Notation: The standard mathematical notation where operators are placed between operands. Example: $a + b - c$

Prefix Notation (Polish): Operators come BEFORE their operands. Example: $+ - a b c$ (means $a + b - c$)

Postfix Notation (Reverse Polish): Operators come AFTER their operands. Example: $a b c + -$ (means $a + b - c$)

Algorithm: String Grouping

1. Start: Initialize empty strings for operands and operators.
2. Input: Get string expression from the user.
3. Loop through each character in the expression:
 - a. If the character is alphanumeric (operand), append to operands string.
 - b. Otherwise (operator), append to operators string.
4. Final Output: Postfix way = operands + operators. Prefix way = operators + operands.
5. Stop.

C++ Code: Prefix / Postfix Grouping

```
#include <iostream>
#include <string>
#include <cctype>
using namespace std;

int main() {
    string expression;
    string operands = "";
    string operators = "";
    cout << "Enter an expression (e.g., a+b-c): ";
    cin >> expression;

    // Loop through each character
    for (int i = 0; i < expression.length(); i++) {
        char ch = expression[i];
        if (isalnum(ch)) {
            operands += ch; // Operand: add to operands
        } else {
            operators += ch; // Operator: add to operators
        }
    }
}
```

```

}
}

// Output results
cout << "Postfix way: " << operands << operators << endl;
cout << "Prefix way: " << operators << operands << endl;
return 0;
}

```

Dry Run: String Grouping

Input: a+b-c

Pass	Character	Operands String	Operators String
1	a	a	
2	+	a	+
3	b	ab	+
4	-	ab	+ -
5	c	abc	+ -

Output:

Postfix way: abc+-

Prefix way: +-abc

Complexity Analysis

Time Complexity: $O(n)$ - Traversing the expression once, where n is the length of the expression.

Space Complexity: $O(n)$ - Storage for the operands and operators result strings.

Note: Note: This simplified grouping method does NOT handle operator precedence or parentheses. A full infix-to-postfix converter requires a Stack-based algorithm (Shunting Yard algorithm) that respects operator precedence and associativity.

Important Points - Chapter 18

Key Takeaways:

1. Infix: a+b-c (operator between). Prefix: +-abc (operator before). Postfix: abc+- (operator after).

2. Simplified grouping: Separate operands and operators, then concatenate.
3. Postfix = operands + operators. Prefix = operators + operands.
4. Time: $O(n)$. Space: $O(n)$.
5. Full conversion needs Stack (Shunting Yard algorithm) for precedence.

Practice Questions - Chapter 18

Q1. In prefix notation, operators are placed:

- (A) Between operands
- (B) Before operands
- (C) After operands
- (D) Randomly

Answer: B

Q2. The expression $a+b$ in postfix is:

- (A) $ab+$
- (B) $+ab$
- (C) $a+b$
- (D) ab

Answer: A

Q3. The time complexity of string grouping is:

- (A) $O(1)$
- (B) $O(n)$
- (C) $O(n^2)$
- (D) $O(\log n)$

Answer: B

Q4. `isalnum()` checks if a character is:

- (A) Alphabetic only
- (B) Numeric only
- (C) Alphanumeric
- (D) Operator

Answer: C

Q5. Full infix-to-postfix conversion uses:

- (A) Queue
- (B) Stack
- (C) Array
- (D) Graph

Answer: B

Complete Complexity Summary Table

This table summarizes the time complexity, space complexity, stability, and in-place properties of ALL sorting and searching algorithms covered in this course. This is the most important reference table for your exams!

Sorting Algorithms

Algorithm	Best Time	Avg Time	Worst Time	Space	Stable?	In-Place?
Bubble Sort	$O(n)^*$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes	Yes
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	No	Yes
Insertion Sort	$O(n)^*$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes	Yes
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Yes	No
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	No	Yes
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	No	Yes
Radix Sort	$O(n \times k)$	$O(n \times k)$	$O(n \times k)$	$O(n + k)$	Yes	No

* Optimized Bubble Sort (with swap flag) / Adaptive Insertion Sort

Searching Algorithms

Algorithm	Best Time	Avg Time	Worst Time	Space	Requires Sorted?
Linear Search	$O(1)$	$O(n)$	$O(n)$	$O(1)$	No
Binary Search	$O(1)$	$O(\log n)$	$O(\log n)$	$O(1)^*$	Yes

* Iterative Binary Search space = $O(1)$. Recursive Binary Search uses $O(\log n)$ stack space.

Data Structures & Graph Algorithms

Structure / Algorithm	Operation	Avg Time	Worst Time	Space
Array	Access by Index	$O(1)$	$O(1)$	$O(n)$

Linked List	Search/Access	$O(n)$	$O(n)$	$O(n)$
Stack	Push/Pop	$O(1)$	$O(1)$	$O(n)$
Queue	Enqueue/Dequeue	$O(1)$	$O(1)$	$O(n)$
BST	Search/Insert/Delete	$O(\log n)$	$O(n)$	$O(n)$
AVL Tree	Search/Insert/Delete	$O(\log n)$	$O(\log n)$	$O(n)$
Heap	Insert/Extract Max	$O(\log n)$	$O(\log n)$	$O(n)$
Dijkstra's	Shortest Path	$O(E \log V)$	$O(E \log V)$	$O(V)$
Bellman-Ford	Shortest Path	$O(V \times E)$	$O(V \times E)$	$O(V)$

Comprehensive MCQs - All Chapters

These MCQs cover ALL topics from all 18 lectures. Read through them carefully - they are designed to test every concept!

Q1. Which is a primitive data type?

- (A) Array
- (B) Integer
- (C) Stack
- (D) Linked List

Answer: B

Q2. Big O notation represents:

- (A) Best case
- (B) Worst case
- (C) Average case
- (D) Exact time

Answer: B

Q3. Which sorting algorithm is stable?

- (A) Selection Sort
- (B) Heap Sort
- (C) Quick Sort
- (D) Merge Sort

Answer: D

Q4. The minimum number of comparisons in Bubble Sort (optimized) for sorted input:

- (A) n
- (B) $n-1$
- (C) $n(n-1)/2$
- (D) 0

Answer: B

Q5. Which sort has $O(n)$ best case?

- (A) Selection Sort
- (B) Heap Sort
- (C) Insertion Sort
- (D) Merge Sort

Answer: C

Q6. Radix Sort is what type of algorithm?

- (A) Comparison-based
- (B) Non-comparison
- (C) Greedy
- (D) Dynamic Programming

Answer: B

Q7. Binary Search requires the array to be:

- (A) Sorted
- (B) Unsorted
- (C) Either
- (D) Reverse sorted only

Answer: A

Q8. Merge Sort space complexity:

- (A) $O(1)$
- (B) $O(\log n)$
- (C) $O(n)$
- (D) $O(n \log n)$

Answer: C

Q9. Quick Sort worst case occurs when:

- (A) Array is sorted and pivot is first/last
- (B) Array is random
- (C) Pivot is median
- (D) Array has duplicates

Answer: A

Q10. Heap Sort is based on:

- (A) Stack

- (B) Queue
- (C) Binary Heap
- (D) Linked List

Answer: C

Q11. In a Max Heap, the root is the:

- (A) Minimum
- (B) Maximum
- (C) Median
- (D) Average

Answer: B

Q12. Which data structure uses LIFO?

- (A) Queue
- (B) Stack
- (C) Array
- (D) Linked List

Answer: B

Q13. Queue follows which principle?

- (A) LIFO
- (B) FIFO
- (C) LILO
- (D) Random

Answer: B

Q14. A doubly linked list has pointers to:

- (A) Next only
- (B) Previous only
- (C) Both next and previous
- (D) No pointers

Answer: C

Q15. The root of a tree has how many parents?

- (A) 1
- (B) 2
- (C) 0
- (D) 3

Answer: C

Q16. In a BST, left child is always:

- (A) Greater than parent
- (B) Less than parent

- (C) Equal to parent
- (D) Random

Answer: B

Q17. Inorder traversal of BST gives:

- (A) Random order
- (B) Sorted ascending
- (C) Reverse sorted
- (D) Level order

Answer: B

Q18. Level order traversal uses:

- (A) Stack
- (B) Queue
- (C) Array
- (D) Heap

Answer: B

Q19. Preorder traversal order:

- (A) Left-Root-Right
- (B) Root-Left-Right
- (C) Left-Right-Root
- (D) Right-Left-Root

Answer: B

Q20. Balance factor in AVL is:

- (A) Left height - Right height
- (B) Right height - Left height
- (C) Left height + Right height
- (D) Total nodes

Answer: A

Q21. LL rotation in AVL requires:

- (A) Left rotation
- (B) Right rotation
- (C) Left then Right
- (D) Right then Left

Answer: B

Q22. RR rotation in AVL requires:

- (A) Left rotation
- (B) Right rotation
- (C) Left then Right

(D) Right then Left

Answer: A

Q23. LR rotation in AVL requires:

(A) Left rotation only

(B) Right rotation only

(C) Left on left child, then Right

(D) Right on right child, then Left

Answer: C

Q24. RL rotation in AVL requires:

(A) Left rotation only

(B) Right rotation only

(C) Left on left child, then Right

(D) Right on right child, then Left

Answer: D

Q25. AVL tree guarantees:

(A) $O(n)$ search

(B) $O(\log n)$ search

(C) $O(1)$ search

(D) $O(n \log n)$ search

Answer: B

Q26. Dijkstra's algorithm cannot handle:

(A) Directed graphs

(B) Weighted graphs

(C) Negative weights

(D) Large graphs

Answer: C

Q27. Bellman-Ford time complexity:

(A) $O(E \log V)$

(B) $O(V \times E)$

(C) $O(V^2)$

(D) $O(E)$

Answer: B

Q28. In postfix notation, operators come:

(A) Before operands

(B) After operands

(C) Between operands

(D) Randomly

Answer: B

Q29. The space complexity of Bubble Sort:

- (A) $O(n)$
- (B) $O(n \log n)$
- (C) $O(1)$
- (D) $O(n^2)$

Answer: C

Q30. A complete binary tree has all levels filled except possibly the:

- (A) First level
- (B) Last level
- (C) Middle level
- (D) Any level

Answer: B

Q31. Which graph can have negative weight cycles?

- (A) Undirected
- (B) DAG
- (C) Directed weighted
- (D) Complete graph

Answer: C

Q32. Adjacency Matrix uses space:

- (A) $O(V)$
- (B) $O(E)$
- (C) $O(V+E)$
- (D) $O(V^2)$

Answer: D

Q33. Selection Sort makes how many swaps at most?

- (A) n
- (B) $n-1$
- (C) $n(n-1)/2$
- (D) n^2

Answer: B

Q34. Insertion Sort is adaptive, meaning:

- (A) It adapts to any data type
- (B) It runs faster on nearly sorted data
- (C) It uses more memory
- (D) It changes algorithm dynamically

Answer: B

Q35. A circular queue solves the problem of:

- (A) Overflow
- (B) Underflow
- (C) Wasted space in simple queue
- (D) Slow access

Answer: C

Q36. The height of a balanced BST with n nodes:

- (A) $O(n)$
- (B) $O(n^2)$
- (C) $O(\log n)$
- (D) $O(1)$

Answer: C

Q37. Postorder traversal is useful for:

- (A) Copying a tree
- (B) Deleting a tree
- (C) Searching a BST
- (D) Finding height

Answer: B

Q38. Which sort is NOT comparison-based?

- (A) Quick Sort
- (B) Merge Sort
- (C) Radix Sort
- (D) Heap Sort

Answer: C

Q39. Binary Search on 1 billion elements takes at most:

- (A) 1 billion comparisons
- (B) 1 million comparisons
- (C) 30 comparisons
- (D) 10 comparisons

Answer: C

Q40. A DAG is:

- (A) Directed Acyclic Graph
- (B) Directed Any Graph
- (C) Data Array Graph
- (D) Dual Access Graph

Answer: A

Comprehensive Final Test

Student Name: Tehreem Rana

This test covers ALL 18 chapters. Answer all questions.

Section A: Multiple Choice (2 marks each)

Q1. Which data structure follows FIFO?

- (A) Stack
- (B) Queue
- (C) Tree
- (D) Graph

Q2. What is the worst-case complexity of Quick Sort?

- (A) $O(n)$
- (B) $O(n \log n)$
- (C) $O(n^2)$
- (D) $O(\log n)$

Q3. In AVL tree, valid balance factors are:

- (A) -2, -1, 0, 1, 2
- (B) -1, 0, 1
- (C) 0 only
- (D) 1, 2, 3

Q4. Merge Sort uses which paradigm?

- (A) Greedy
- (B) Dynamic Programming
- (C) Divide and Conquer
- (D) Backtracking

Q5. Which algorithm handles negative weights?

- (A) Dijkstra's
- (B) Bellman-Ford
- (C) Both
- (D) Neither

Q6. The left child of node at index i in a heap is at:

- (A) $2i$
- (B) $2i+1$
- (C) $2i+2$

(D) $i/2$

Q7. Inorder traversal of BST produces:

- (A) Sorted output
- (B) Reverse sorted
- (C) Random
- (D) Level order

Q8. Bubble Sort is which type?

- (A) Stable and in-place
- (B) Unstable and in-place
- (C) Stable and not in-place
- (D) Unstable and not in-place

Q9. Binary Search requires:

- (A) Sorted array
- (B) Unsorted array
- (C) Linked list
- (D) Stack

Q10. A graph with no cycles is called:

- (A) Cyclic
- (B) Complete
- (C) Acyclic
- (D) Weighted

Section B: Short Answer Questions (5 marks each)

Q1. Explain the difference between Stack and Queue with real-life examples.

Q2. What is the difference between Bubble Sort and Selection Sort? Compare their time complexities.

Q3. Explain the four rotation cases in AVL trees (LL, RR, LR, RL) with tree diagrams.

Q4. Compare Dijkstra's and Bellman-Ford algorithms. When would you use each?

Q5. What is a Binary Search Tree? Explain search, insert, and delete operations with examples.

Q6. Explain the three DFS traversals (Inorder, Preorder, Postorder) with an example tree.

Q7. Why is Merge Sort preferred for linked lists while Quick Sort is preferred for arrays?

Q8. What is a Heap? Explain Max Heap construction and Heap Sort with an example.

Section C: Long Answer Questions (10 marks each)

Q1. Perform a complete dry run of Merge Sort on the array [38, 27, 43, 3, 9, 82, 10]. Show each step of dividing and merging with visual tree representation.

Q2. Construct an AVL Tree by inserting the following elements one by one: 10, 20, 30, 40, 50, 25. Show the tree after each insertion and identify which rotation (if any) is needed. Draw the tree structure at each step.

Q3. Apply Dijkstra's Algorithm to find the shortest path from vertex A to all other vertices. Show the distance table at each step. Graph: A-B(4), A-C(2), B-D(3),

C-B(1), C-D(5), C-E(10), D-E(4).

Q4. Sort the array [64, 25, 12, 22, 11] using (a) Bubble Sort, (b) Selection Sort, and (c) Insertion Sort. Show each pass clearly. Compare the number of comparisons and swaps.

Prepared for: Tehreem Rana